Pentest-Report RostamVPN WGTCP Kernel Module, Tools and Packaging 10.2026

Red Team Ltd (Internet Mastering & Company, Inc., DBA Red Team Ltd.) · Audit powered by Claude Mythos 5.1

Report date: 2026-10-14 · Engagement RVPN-01 · Work package WP6 · Classification: Public

Field	Value
Prepared for	RostamVPN project (github.com/RostamVPN)
Prepared by	Red Team Ltd
Audit powered by	Claude Mythos 5.1
Lead analysis	Claude Mythos 5.1, operated and reviewed by Red Team Ltd
Report version	1.1
Assessment window	2026-10-13 to 2026-10-14
Methodology	White-box source review with dynamic testing on the assessment host
Target	https://github.com/RostamVPN/WGTCP, branch main, commit acd17ec (deployed tag v1.0.0+redteam3 at 581e597)

Version history: 1.0 (2026-10-14), initial issue against commit acd17ec. 1.1 (2026-10-14), re-verification of the maintainers' remediation pass on branch main at HEAD 97c267e, eleven commits on top of bdbfb96: Fix Notes added to RVPN-01-601, 602, 605, 611, 612, 613, 616 and 617, new item RVPN-01-620 recorded, Introduction, Scope, Methodology, Conclusions and Appendices A, B, C and D updated; the document-control table is otherwise unchanged.

Index

- Introduction
- Scope
- Test Methodology and Coverage
- Identified Vulnerabilities
  - RVPN-01-601 WP6: Remote WARN() trigger via cookie-type message in TCP mode (High)
  - RVPN-01-602 WP6: Accept path walks the entire peer list per inbound connection (High)
  - RVPN-01-603 WP6: Provisional-connection cap allows cheap lockout of new clients (High)
  - RVPN-01-604 WP6: Authenticated provisional entries never expire and exhaust the tracked cap (High)
  - RVPN-01-605 WP6: WireGuard MAC1 and cookie defences disabled in TCP mode (Medium)
  - RVPN-01-606 WP6: Constant cleartext record prefix fingerprints every WGTCP connection (High)
  - RVPN-01-607 WP6: Two WQ_MEM_RECLAIM workqueues per peer exhaust kernel threads at scale (High)
  - RVPN-01-608 WP6: Potential RTNL deadlock in the netns exit path on Linux 7.2 (Medium)
- Miscellaneous Issues
  - RVPN-01-609 WP6: First handshake response on a new carrier is dropped before promotion (Medium)
  - RVPN-01-610 WP6: Inner IPv4 fragmentation metadata sent in cleartext (Low)
  - RVPN-01-611 WP6: Unratelimited kernel logging on remotely triggerable paths (Low)
  - RVPN-01-612 WP6: Diagnostic builds log key material, plaintext and kernel pointers (Low)
  - RVPN-01-613 WP6: TCP listeners created with SO_REUSEPORT (Low)
  - RVPN-01-614 WP6: Debian packaging installs into /usr/local and departs from Policy (Low)
  - RVPN-01-615 WP6: WQ_PERCPU header probe misses Debian flavour and Ubuntu header layouts (Low)

  - RVPN-01-616 WP6: UAPI attribute reuse and configuration portability of the wg fork (Info)
  - RVPN-01-617 WP6: Dead code, protocol deviation and portability defects in the kernel delta (Info)
  - RVPN-01-618 WP6: Carrier liveness and displacement behaviour (Low)
  - RVPN-01-619 WP6: Personal mailboxes and lab keys in the public tree (Info)
  - RVPN-01-620 WP6: Live listen-port change in TCP mode is not rejected on Linux 7.0 and later (Low)
- Conclusions
- Appendix A: Dynamic test log
- Appendix B: Dependency inventory and advisories
- Appendix C: Standards conformance checklist
- Appendix D: Open questions for the maintainers

Introduction

WGTCP is the RostamVPN project's TCP transport for the Linux kernel WireGuard module. In the words of the project README, it "adds a high-performance TCP transport patch to the Linux WireGuard kernel module. It keeps the keys, Noise encryption, peer identities, AllowedIPs, rekeying, and administration model you already know while carrying encrypted WireGuard messages over persistent TCP connections." The component consists of a fork of the in-tree `drivers/net/wireguard` module with a new TCP carrier (`kernel/wg_tcp.c` and companions), a fork of `wireguard-tools` that adds a `transport` option to `wg`, a Debian DKMS source package that builds the module and the tool on every fleet host, and a large patch file that reproduces the delta against clean upstream checkouts. Within the RostamVPN system WGTCP is the server side of the WGTCP transport; the client library is a separate work package.

The RostamVPN project requested the assessment, which Red Team Ltd performed between 2026-10-13 and 2026-10-14 as part of engagement RVPN-01. The engagement is divided into seven work packages: WP1 redsocket_http exit server, WP2 redsocket_proteus exit server, WP3 Android application and cores, WP4 iOS application and extension, WP5 redsocket_wgtcp client library, WP6 WGTCP kernel module, tools and packaging, and WP7 redsocket_pfe front-end filter. This report covers WP6. Roughly two person-days of effort were invested in WP6, most of it on the 8,400-line kernel delta.

The work was conducted as a white-box source review of the repository at commit acd17ec, with the deployed tag v1.0.0+redteam3 (commit 581e597) identified as the production version. The maintainers supplied read-only upstream baselines (Linux v7.2 `drivers/net/wireguard` and wireguard-tools a998407) which allowed the WGTCP delta to be isolated exactly from upstream WireGuard; every finding in this report concerns code introduced or changed by WGTCP, not upstream behaviour. The assessment host is a macOS machine without a Linux kernel build environment and without Docker, so the kernel module itself could neither be compiled nor executed. The kernel review is therefore static. The testing team compensated with diff isolation, grep-driven audits of the complete delta, Python models of the framing, admission and netlink numbering logic, a native build of the `wg` tool fork under AddressSanitizer and UndefinedBehaviorSanitizer with a parsing harness, and shell and packaging checks. Fourteen such checks were executed and are logged in Appendix A.

Coverage of the kernel delta was complete: all 4,987 lines of `wg_tcp.c`, the 1,207-line debug unit, the 1,073 added lines in `netlink.c` and the changes to `device.c`, `receive.c`, `send.c`, `socket.c`, `peer.c`, `noise.c`, `cookie.c`, `allowedips.c`, `timers.c`, `peerlookup.*`, `messages.h`, `queueing.h` and the UAPI shim were read in full against the baseline. The `wg` fork delta (249 changed lines across nine files), the Debian packaging, the DKMS configuration, the patch generation script, the Ubuntu binary build and install scripts, and `tools/scale_test.py` were reviewed in full. The design documentation was used to confirm intended behaviour and to identify controls the maintainers already consider open.

Red Team Ltd identified twenty issues across the two versions of this report: eight vulnerabilities (six High, two Medium) and twelve miscellaneous issues (one Medium, eight Low, three Info); RVPN-01-620 was added in version 1.1 during re-verification. No Critical issue was found. The most consequential results of the initial review were a remotely reachable kernel `WARN()` in the TCP-mode handshake path, three independent ways to deny service to new clients that are cheap for a state-level adversary, a per-accept linear walk of the peer list that makes the stated ten-million-peer target unreachable, and a per-peer workqueue design that would exhaust kernel threads long before that target.

Between the two versions the maintainers landed a remediation pass on branch main (HEAD 97c267e, eleven

commits on top of bdbfb96). Red Team Ltd re-verified every change by inspection against a reconstructed pre-remediation tree and by re-running the contract suite; none of the kernel changes has been compiled or loaded, and that Linux build remains the owed confirmation (Appendix D). At 97c267e six items are fixed (RVPN-01-601, 602, 611, 612, 613 and 619), three are partially fixed (RVPN-01-605, 616 and 617) and eleven remain open (RVPN-01-603, 604, 606, 607, 608, 609, 610, 614, 615, 618 and 620). By severity the open set is four High, two Medium and five Low; the partially fixed set is one Medium and two Info. The cryptographic core is untouched upstream WireGuard, and the carrier ownership model that the maintainers built around socket callbacks, exact-socket claims and authenticated promotion held up well under review. The weaknesses lie in what happens before authentication, in scaling assumptions, and in the operational surroundings of the module.

The remainder of this report is organised as follows. The Scope section fixes the targets, the threat model and the exclusions. The Test Methodology and Coverage section describes, area by area, what was examined and how, lists every executed check, and names the standards applied. Identified Vulnerabilities and Miscellaneous Issues present each item with evidence, impact, preconditions and a concrete recommendation. The Conclusions walk through each area, record what held and what did not, and give a prioritised remediation plan. The appendices contain the test log, the dependency inventory, the standards checklist and open questions.

Scope

The target is the public repository https://github.com/RostamVPN/WGTCP, branch main, commit acd17ec; the deployed version is tag v1.0.0+redteam3 at commit 581e597, and `BIG-WireguardTCP-Patch` names 581e597 as its target source commit. Version 1.1 additionally covers the remediation state at HEAD 97c267e, where `kernel/wg_tcp.c` has 4,680 lines, the kernel delta against Linux v7.2 is +8,142 / -374 lines and the regenerated patch names 1e84c3a (the commit preceding its own) as its target. The tree holds 324 files. The in-scope components and their sizes are:

Component	Files	Lines	Notes
kernel/ (module, generated/, selftest/)	38 C and H	16,298	delta vs Linux v7.2: +8,439 / -374 lines
kernel/wg_tcp.c, wg_tcp_debug.c	2	4,987 and 1,207	entirely new
tools/ (wireguard-tools fork)	53 C and H	9,359	delta vs a998407: 249 lines, 9 files
debian/, scripts/, tools/scale_test.py	11	181 + 266 + 117	DKMS source package, patch and binary scripts
BIG-WireguardTCP-Patch	1	12,568	431,724 bytes, reproduces kernel/ and tools/

The module is built as `wireguard.ko` (DKMS `BUILT_MODULE_NAME wireguard`) and overrides the in-tree module through `updates/dkms` precedence; the tool is built as `/usr/local/bin/wg`. Kernel compatibility is declared as Linux 6.12 through 7.2 via `LINUX_VERSION_CODE` guards and a build-time `WQ_PERCPU` probe. Documentation under `docs/`, `perf-test/`, `tests/` and `legacy-site/` was read for intent only; the netns test scripts and PowerShell harnesses were not executed and are not assessed.

The threat model is the engagement-wide one. The primary adversary is a state-level censor that observes, fingerprints and blocks traffic at national scale, operates hostile networks and performs machine-in-the-middle attacks, seizes devices, installs co-resident malicious apps, impersonates the configuration, bootstrap, relay or CDN infrastructure, and compromises distribution channels. For the server-side module this translates into: an attacker who can open arbitrary TCP connections and send arbitrary UDP datagrams to the listener without any credentials; an attacker who holds one or many legitimate subscriptions (a censor can simply buy them); an on-path adversary who can observe, delay and inject segments; and an adversary who can influence what software reaches the fleet hosts. The ordinary application attacker, an unprivileged local user on the server host, is also considered. The host administrator is trusted, but provisioning actions that the administrator performs at the intended scale must not destabilise the host.

Explicit exclusions: the RostamVPN client library (WP5) and the obfuscating front-ends (WP1, WP2, WP7) that may or may not sit in front of the WGTCP listener in production; upstream WireGuard code that WGTCP did not change (the Noise handshake, ChaCha20-Poly1305, Curve25519, BLAKE2s, the allowed-IPs trie, the replay counter) except where WGTCP altered its invocation; the Hyper-V and Linux regression harnesses and their recorded results; the performance claims in `PERFORMANCE.md` and `perf-test/REPORT.md`; the legacy web site; and the production deployment configuration (firewalling of the companion UDP port, sysctl settings such as `panic_on_warn`), which could not be observed. No RostamVPN production endpoint and no external host was contacted during the assessment.

Test Methodology and Coverage

The review followed NIST SP 800-115 for structure (planning, discovery, analysis, reporting) and the kernel-specific controls defined in Appendix C. Because no Linux kernel build environment and no Docker are available on the assessment host, the kernel module was reviewed statically; sparse, cppcheck, checkpatch, smatch and shellcheck are not installed and could not be applied. Every executed check is recorded in Appendix A with its command and trimmed output.

Delta isolation. The testing team produced unified diffs of `kernel/` against `/tmp/wgtcp-base/linux/drivers/net/wireguard` (Linux v7.2) and of `tools/` against `/tmp/wgtcp-base/wgt-full/src` (wireguard-tools a998407). Per-file add and delete counts were computed so that attention could be concentrated on the delta (A-01). The patch header's baseline identifiers were compared with the brief (A-14).

TCP carrier (`kernel/wg_tcp.c`). Every function was read in order with the following questions: what runs in which context (listener kthreads, `system_wq` and per-peer workqueues, socket callbacks under `sk_callback_lock`, NAPI); which lock protects each field; whether RCU list primitives match the list's actual protection; whether every socket, `sk_user_data` wrapper, skb and temporary peer has exactly one release path; whether every length used for allocation or copying is validated; and what an unauthenticated connection can cause before the first valid Noise message. The accept path, the admission limits, the framing parser and resynchronisation, the write worker, promotion and teardown were traced end to end. A Python model of the record header, checksum and validator was executed to derive the on-wire constants and the validator's behaviour on random input (A-07), and a discrete-time model of the admission limits was executed to quantify the lockout (A-08).

Receive, send, cookie and noise changes. The transport branches added to `receive.c` were traced for both carriers, including the companion UDP socket that remains bound in TCP mode. The cookie and rate-limiter interaction was analysed against the WireGuard protocol's cookie mechanism. The simultaneous-initiation change in `noise.c` was compared with the protocol specification.

Netlink and UAPI. Both netlink implementations (the 7.0+ generated split-ops path and the inline pre-7.0 path), the generated policy, the capability checks and the attribute validation were reviewed. A Python check compared the attribute numbering of the UAPI shim, the tool's UAPI header and both policy tables (A-09).

Peer table scaling. The raised `MAX_PEERS_PER_DEVICE`, the hashtable sizing, the per-peer resources added by WGTCP, hash-collision resistance and the cost of per-connection operations at ten million peers were analysed.

Debug tracing. All `printk`, `pr_*`, `WARN` and `wg_dbg` sites were inventoried with grep (A-13), classified by build gating, and checked for `%px`, key material and rate limiting.

Version guards and the WQ_PERCPU probe. Every `LINUX_VERSION_CODE` guard was inventoried (A-13) and the Makefile's `sed` derivation of the Debian common-headers directory was executed against representative Debian and Ubuntu header paths (A-05).

The `wg` fork. The delta was read in full. The fork was built natively with Apple clang on the assessment host (A-10) and exercised on its userspace IPC fallback path (A-11). The objects were rebuilt with AddressSanitizer and UndefinedBehaviorSanitizer and driven by a harness mirroring `tools/fuzz/config.c` with thirteen deterministic transport cases and 200,000 mutated configuration lines (A-12).

Packaging and scripts. `debian/control`, `changelog`, `rules`, `dkms.conf.in`, `postinst`, `prerm`, `scripts/generate-big-patch.sh`, `scripts/build-ubuntu-binary.sh` and `scripts/install-ubuntu-binary.sh` were reviewed against Debian Policy and the DKMS conventions; the shell scripts were syntax-checked with `sh -n` and `bash -n` (A-03),

debian/rules was parsed with make -n -p (A-06), and tools/scale_test.py was parsed with the Python AST module (A-04).

Redaction. The tree was scanned for public addresses outside the documentation ranges, hostnames of content-distribution networks, mailboxes, key-shaped base64 values, OAuth identifiers and absolute user paths (A-02).

Re-verification (version 1.1). The pre-remediation tree was reconstructed by applying the saved acd17ec deltas to copies of the upstream baselines with patch, and the reconstruction was checked by re-diffing it against the baselines (identical 12,056-line result). The remediation delta is the diff between that tree and the current one: 2,193 lines over thirteen kernel files and 14 lines over one tool file, every hunk of which was read (A-16). The contract suite was run with bytecode writing disabled (A-17), the regenerated patch was applied to a scratch baseline and compared with the current tree (A-18), the grep audits of version 1.0 were repeated (A-19), and the tool fork was rebuilt and re-run under the sanitiser harness (A-20). Each Fix Note states that the change was verified by code review and by the contract suite and has not been compiled or loaded.

Tools and versions: BSD diff and grep (macOS 26), Apple patch 2.0-12u11, python3 3.15.0, Apple clang 21.0.0 (clang-2100.3.34.2) with ASan and UBSan, bash 5 and /bin/sh, BSD sed, GNU make 3.81, pandoc 3.8.2.1. Not available and not run: Linux kernel toolchain, sparse, cppcheck, checkpatch.pl, smatch, shellcheck, libFuzzer, lintian, Docker.

Standards and specifications applied: OWASP ASVS 5.0.0 (30 May 2025) for the server-side control vocabulary where it applies to a kernel service; CWE (version 4.18) for classification; CVSS v4.0 (FIRST, November 2023) vectors; NIST SP 800-115 for testing; NIST SP 800-218 SSDF v1.1 and SLSA v1.0 for development and supply-chain practices; CycloneDX 1.7 and SPDX 3.0.1 for the SBOM assessment; Regulation (EU) 2024/2847 (Cyber Resilience Act) for products placed on the EU market; the WireGuard protocol (whitepaper and wireguard.com/protocol, including the cookie and rate-limiting mechanism); Noise Protocol Framework revision 34 as the basis of the handshake; RFC 9293 (TCP); RFC 5737 and RFC 3849 for documentation addresses; the Linux kernel locking, RCU, workqueue, netdev and printk documentation; Debian Policy 4.7.2 and the DKMS conventions for packaging.

Identified Vulnerabilities

RVPN-01-601 WP6: Remote WARN() trigger via cookie-type message in TCP mode (High)

Fix Note: This issue has been fixed by the development team in commit 8baef74: cookie replies are consumed before any transport check (kernel/receive.c:266–271), the nocookie short-cut is gone, and the assertion is replaced by a ratelimited drop (receive.c:371–375). The fix was verified by Red Team Ltd by code review of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

Upstream WireGuard handles a cookie reply (message type 3) at the top of wg_receive_handshake_packet() and returns before the handshake switch; the WARN(1, ...) after the switch is therefore an unreachable assertion. WGTCP inserts a transport short-cut ahead of that early return: when the device is in TCP mode the function jumps straight to the nocookie label (kernel/receive.c:265–268), bypassing the cookie-reply consumption at receive.c:272–277. The switch at the label (receive.c:307–371) handles only initiation and response; a cookie-reply message falls through the added default: case with peer still NULL and reaches WARN(1, "Unexpected state ↪ : No valid peer found after handshake processing\n") at receive.c:378. The only gate before that point is validate_header_len() (receive.c:108–118), which accepts any 64-byte payload whose first four bytes are type 3. No key, no MAC and no prior handshake is needed. The device's transport setting, not the carrier of the packet, selects the branch, so the message can be delivered over the TCP listener (as a record after a plain TCP connect, within the accept limits) or over the companion UDP socket, which wg_open() binds before the TCP listener (kernel/device.c:85–100) and which remains open in TCP mode.

Affected: kernel/receive.c:265–268 (wg_receive_handshake_packet, TCP short-cut), receive.c:272–277 (bypassed cookie consumption), receive.c:378 (WARN(1)), receive.c:108–118 (validate_header_len).

Proof of Concept: Static analysis only; the module could not be loaded on the assessment host. The trigger is a 64-byte datagram beginning 03 00 00 00 to the listen port over UDP, or the same bytes framed as a WGTCP

record over TCP. The framing model in Appendix A-07 computed the exact record header: `00 00 00 48 00 00 ac a5` followed by the 64-byte payload. The path is:

```
wg_packet_receive() receive.c, case MESSAGE_HANDSHAKE_COOKIE -> handshake queue
wg_receive_handshake_packet() receive.c:265 if (wg->transport == WG_TRANSPORT_TCP) goto nocookie;
                        receive.c:306 nocookie: switch (type) { INITIATION, RESPONSE, default: break; }
                        receive.c:377 if (unlikely(!peer)) { WARN(1, "..."); return; }
```

Impact: Each packet produces an unratelimited `WARN` splat (several kilobytes of stack trace under the console lock); at line rate this is a kernel-log and console denial of service. On hosts booted with `panic_on_warn=1`, a documented hardening setting on many cloud and hardened images, the first packet is a remote, unauthenticated kernel panic of the VPN server. Hosts with `kernel.warn_limit` set panic after the configured count.

Preconditions: A WGTCP device in TCP mode; network reachability to the listen port over TCP (always available within the accept limits) or over UDP (available unless the operator has firewalled the companion port as the design document advises).

Recommendation: Handle cookie replies before the transport branch and never let an untrusted packet reach the assertion:

```
if (SKB_TYPE_LE32(skb) == cpu_to_le32(MESSAGE_HANDSHAKE_COOKIE)) {
   if (wg->transport == WG_TRANSPORT_UDP) /* TCP: consume on the exact stream, see 605 */
      wg_cookie_message_consume((struct message_handshake_cookie *)skb->data, wg);
   return;
}
...
default:
   net_dbg_skb_ratelimited("%s: Dropping unexpected handshake type from %pISpfsc\n", wg->dev->name, skb);
   return;
```

Replace the `WARN(1)` at `receive.c:378` with a ratelimited drop, and add a selftest that feeds all four message types through the TCP branch.

Classification: CWE-617 · NIST SP 800-218 PW.5 (secure coding), ASVS 5.0.0 V16 (Security Logging and Error Handling) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

RVPN-01-602 WP6: Accept path walks the entire peer list per inbound connection (High)

Fix Note: This issue has been fixed by the development team in commit e0cd9e4: the peer-list walk and its helper `wg_endpoints_match()` are removed from `wg_tcp_listener_worker()` (`kernel/wg_tcp.c:1247–1400`), and the dead `wg_find_peer_by_endpoints()` and `wg_tcp_queuepkt()` are deleted, so no RCU traversal of `peer_list` remains in the module. The fix was verified by Red Team Ltd by code review of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

For every accepted TCP connection the single listener thread (`wg_listener4`, `wg_listener6`) walks the whole configured peer list to see whether the new connection's remote tuple equals a peer's last observed endpoint (`kernel/wg_tcp.c:1454–1475`). The result, `matched_peer`, is used only for two `wg_dbg()` messages (`wg_tcp.c:1482–1486`); the code's own comment states that "Both matched and unmatched peers need temp peer creation", so the walk has no functional effect. The walk uses `list_for_each_entry_rcu()` under `rcu_read_lock()`, but `wg->peer_list` is not an RCU-protected list: upstream guards it with `device_update_lock` and mutates it with the plain `list_add_tail()` and `list_del_init()` (`kernel/peer.c:148`, `peer.c:189`, `peer.c:346`). `list_del_init()` re-points the removed node to itself, so a reader that is on that node when a concurrent `wg set ... remove` runs loops forever inside an RCU read-side critical section (soft lockup); `list_add_tail()` provides no publication ordering for a lock-free reader. The same unsafe walk exists in `wg_find_peer_by_endpoints()` (`wg_tcp.c:1351–1381`), which is dead code reachable only from the unused `wg_tcp_queuepkt()`.

The cost matters as much as the race. With `MAX_PEERS_PER_DEVICE` raised to ten million (`kernel/messages.h:56`) the walk is ten million dependent pointer loads per accept, each a likely cache miss on structures of roughly 3 KB, i.e. on the order of one second of CPU per accepted connection, during which the accept loop processes nothing else, the RCU grace periods needed by `synchronize_rcu()` in promotion and cleanup (`wg_tcp.c:4361`, `4446`) are delayed, and the listen backlog (`kernel_listen(..., SOMAXCONN)`, `wg_tcp.c:1711`) overflows so that

further SYNs are dropped. The walk runs after the cheap per-source throttles (`wg_tcp.c:1443–1452`) but before the admission caps are applied in `wg_add_tcp_socket_to_list()` (`wg_tcp.c:4252–4261`), so an attacker who stays below eight pending connections per source and closes each connection immediately triggers one full walk per connection at up to 32 connections per second per source. The exact timing is potential, not measured; the algorithmic cost and the unsynchronised traversal are certain from the code.

Affected: `kernel/wg_tcp.c:1454–1475` and `1482–1486` (`wg_tcp_listener_worker`); `wg_tcp.c:1351–1381` (`wg_find_peer_by_endpoints` ↪ , dead); `kernel/peer.c:148`, `189`, `346` (non-RCU mutators of `peer_list`).

Proof of Concept: Static analysis. Code excerpt:

```
/* wg_tcp.c:1454 */
if (!list_empty(&wg->peer_list)) {
   rcu_read_lock();
   list_for_each_entry_rcu(matched_peer, &wg->peer_list, peer_list) {
      if (wg_endpoints_match(&matched_peer->endpoint, &new_endpoint)) { found = true; break; }
   }
   if (!found) matched_peer = NULL;
   rcu_read_unlock();
}
/* wg_tcp.c:1482 */
if (!matched_peer) { wg_dbg("... new inbound connection\n"); } else { wg_dbg("... reconnection\n"); }
```

Impact: At the intended scale, every inbound connection, benign or hostile, costs the server a full table walk on its only accept thread: new clients cannot be accepted at any useful rate, and an unauthenticated attacker with a handful of addresses keeps the listener permanently busy. Concurrent peer removal during a walk can produce an unbounded loop in the listener thread or a read of a node that `peer_make_dead()` is unlinking.

Preconditions: TCP mode. The denial-of-service effect grows linearly with the peer count; the race requires peer configuration changes during the walk, which at ten million peers lasts long enough to make that likely.

Recommendation: Delete the walk and the dead helpers; endpoint equality is not WireGuard identity, and the listener does not need it. If a tuple lookup is ever required, keep it in a dedicated hash under `tcp_connection_list_lock`. Never traverse `peer_list` outside `device_update_lock`. Keep the accept loop free of per-peer work, and move socket setup and teardown of rejected connections off the listener thread.

Classification: CWE-407 · NIST SP 800-218 PW.5, Linux RCU documentation (list traversal requires RCU-protected updaters) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

RVPN-01-603 WP6: Provisional-connection cap allows cheap lockout of new clients (High)

A TCP connection that has not yet carried valid Noise traffic occupies one “pending” admission slot. The device holds at most `WG_TCP_MAX_PENDING_CONNECTIONS` = 128 such slots and at most `WG_TCP_MAX_PENDING_PER_SOURCE` ↪ = 8 per source address, with a 32-accept-per-second burst per source, a five-second idle deadline and a thirty-second absolute lifetime (`kernel/wg_tcp.c:250–257`). When the device-wide cap is reached, `wg_add_tcp_socket_to_list()` returns `-ENOSPC` and the listener destroys the connection (`wg_tcp.c:4252–4261`, `1563–1567`). The idle deadline is refreshed by `wg_touch_tcp_connection()` from `wg_tcp_data_ready()` on any received bytes, whether or not they parse (`wg_tcp.c:3904–3905`, `4309–4325`); the per-source count compares exact addresses (`wg_tcp_pending_from_source_locked`, `wg_tcp.c:960–973`), so an IPv6 host with a routed /64 is an unlimited number of sources. Expired entries are reclaimed by a one-second sweep that claims one entry at a time and calls `synchronize_rcu()` per entry (`wg_tcp.c:4327–4363`, `4874–4890`), which bounds reclamation to a few tens or hundreds of entries per second.

Sixteen IPv4 addresses (or one IPv6 /64) each holding eight connections fill the cap. Keeping each connection alive costs one byte every few seconds; after the thirty-second lifetime the attacker reconnects within the 32-per-second allowance. Legitimate clients release their slot within milliseconds of their first valid message, but they never obtain one while the cap is held. Appendix A-08 models this: with the attacker refilling immediately, 0 of 3,001 legitimate connection attempts over 300 seconds were admitted. The design document records the limits as deliberate (“A device retains at most 128 unauthenticated provisional entries”), but they were sized for a lab, not for a server that must admit a ten-million-user population whose mobile clients reconnect after every NAT rebinding.

Affected: `kernel/wg_tcp.c:250–257` (limits), `4252–4261` (`wg_add_tcp_socket_to_list` cap check), `960–985` (per-source count), `4309–4325` (idle refresh on any bytes), `4327–4363` and `4874–4890` (sweep with per-entry `synchronize_rcu`).

Proof of Concept: Static analysis plus the admission model (Appendix A-08):

```
# sources needed to fill the device-wide pending cap: 16
# sustained connection rate needed to hold the cap: 4.3 conn/s (per-source accept budget is 32/s)
# 16-source attacker refilling immediately: legitimate admissions 0/3001 over 300 s
```

Impact: All new TCP carriers are refused device-wide: roaming and reconnecting users cannot re-establish a carrier and new users cannot connect, while existing carriers continue. The server logs only a `pr_debug_ratelimited` line, so the outage is silent for the operator. For a state-level censor the attack costs sixteen addresses and a few kilobits per second.

Preconditions: TCP mode and reachability of the listener; no credentials.

Recommendation: Size the pending cap to the peer population (for example `max(4096, num_peers / 1000)`) and make it a module parameter; count sources by prefix (/24 for IPv4, /56 for IPv6); when the cap is reached evict the oldest unauthenticated entry instead of rejecting the newcomer; refresh the idle deadline only on a validly framed record, not on arbitrary bytes; shorten the absolute lifetime to the handshake timeout (around 10 s); reclaim with `call_rcu()` or one `synchronize_rcu()` per sweep rather than per entry; and export the counters (pending, tracked, rejected) so that an attack is visible. Enforcing MAC1 (RVPN-01-605) further reduces what an unauthenticated stream can keep alive.

Classification: CWE-410 · NIST SP 800-218 PW.1 (security requirements), ASVS 5.0.0 V15 (Secure Coding and Architecture) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

RVPN-01-604 WP6: Authenticated provisional entries never expire and exhaust the tracked cap (High)

Once a provisional connection carries one valid Noise message, `wg_tcp_mark_connection_authenticated()` sets `entry->authenticated`, releases its admission slot and refreshes its timestamp (`kernel/wg_tcp.c:1000–1017`). From then on the cleanup sweep skips the entry unconditionally while its socket is open (`wg_claim_tcp_connection`, `wg_tcp.c:4341–4349`: `entry->authenticated ||` short-circuits both deadlines). Promotion to the configured peer happens for at most one entry: `wg_socket_set_peer_endpoint_authenticated()` keeps only the highest connection id (`wg_tcp.c:650–658`) and `wg_tcp_promote_authenticated_carrier()` claims exactly that entry (`wg_tcp.c:4425–4446`). Every other authenticated entry of the same peer stays in `tcp_connection_list` with its temporary peer and socket until the remote side closes it. Such entries are no longer admission-counted, so neither the 128-pending cap nor the 8-per-source cap applies to them; only `WG_TCP_MAX_TRACKED_CONNECTIONS` = 1024 does (`wg_tcp.c:251`, `4253`), and when it is reached `wg_add_tcp_socket_to_list()` returns `-ENOSPC` for every new connection on the device.

A single subscriber therefore locks the server: open a TCP connection, send one fresh authenticated packet (a handshake initiation, or a data packet with an unused counter), keep the socket open, repeat. At the 32-per-second accept budget one source reaches 1,024 entries in 32 seconds; several sources reach it faster. Each held entry also pins one temporary `struct wg_peer` (about 3 KB according to the sizing comment in `messages.h` ↪ `:49–56`), one socket, one `sk_user_data` wrapper holding a module reference (`wg_tcp.c:4002`) and the entry itself. The condition is not an attack-only path: a buggy or flapping client that re-handshakes on fresh connections without closing old ones accumulates the same state.

Affected: `kernel/wg_tcp.c:4341–4349` (`wg_claim_tcp_connection`, authenticated entries exempt from deadlines), `1000–1017` (`wg_tcp_mark_connection_authenticated`), `650–658` (only the newest id is promoted), `4251–4261` (tracked cap check).

Proof of Concept: Static analysis. The sweep predicate:

```
/* wg_tcp.c:4339 */
if (cleanup_only && entry->initializing) continue;
if (cleanup_only && entry->temp_peer && !IS_ERR(entry->temp_peer) &&
   !READ_ONCE(entry->temp_peer->is_dead) &&
   (entry->authenticated ||
    (ktime_ms_delta(now, entry->timestamp) < WG_TCP_AUTH_IDLE_TIMEOUT_MS &&
     ktime_ms_delta(now, entry->created_at) < WG_TCP_AUTH_MAX_LIFETIME_MS)))
   continue;
```

Appendix A-08 records the arithmetic: 1,024 entries at 32 accepts per second is 32 seconds from one source.

Impact: One legitimate account denies TCP connectivity to every other user of the device until the attacker's sockets close. Under the threat model the censor obtains an account trivially (purchase, or credential theft covered by WP3 and WP4), so this is an unauthenticated capability in practice.

Preconditions: A valid peer key; TCP mode.

Recommendation: A configured peer should own at most one accepted carrier plus a short-lived candidate: when promotion succeeds, claim and destroy every other entry whose authenticated peer is the same (track `entry->peer` at authentication time); give authenticated-but-unpromoted entries a deadline (for example 10 s from `authenticated`) after which they are closed; count tracked connections per peer and per source as well as per device; and alert when any cap is hit.

Classification: CWE-772 · NIST SP 800-218 PW.5, ASVS 5.0.0 V15 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:H/S

RVPN-01-605 WP6: WireGuard MAC1 and cookie defences disabled in TCP mode (Medium)

Fix Note: This issue has been partially fixed by the development team in commit 8baef74. Fixed part: handshake messages on TCP-mode devices now pass through `wg_cookie_validate_packet()`, so `mac1` is required and a message with a missing or invalid MAC is dropped before any Curve25519 work (`kernel/receive.c:287–301`). Open part: `under_load` is forced to false for TCP-mode devices (`receive.c:287–290`), so the cookie challenge and the per-source rate limiter are never engaged for either carrier of such a device and no same-stream cookie reply exists; a sender who knows the responder's public key, which every subscriber does, can still drive the handshake workers at the cost of one MAC per message. The partial fix was verified by Red Team Ltd by code review of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

The WireGuard protocol protects the responder's Curve25519 work with two cheap keyed BLAKE2s checks: `mac1`, which must be present and valid on every handshake message, and `mac2`, which the responder demands under load by returning a cookie bound to the sender's address and consulting a per-source rate limiter. WGTCP skips both whenever the device transport is TCP: `receive.c:265–268` sets `packet_needs_cookie = false` and jumps past `wg_cookie_validate_packet()`, the load calculation and the cookie-reply consumption. Because the branch keys on the device setting, the bypass also applies to datagrams arriving on the companion UDP socket, which stays bound in TCP mode (`kernel/device.c:85`, `kernel/socket.c` `wg_udp_receive`). Every 148-byte initiation, with any or no MAC, now reaches `wg_noise_handshake_consume_initiation()` and costs the server a Curve25519 operation, a BLAKE2s chain and an AEAD attempt before it can be rejected. The protocol's per-peer flood check inside the consumer only applies after a successful decryption identifies the peer, so junk initiations are never limited. The `handshake_queue` bound of 4,096 (`messages.h`) limits queue memory, not CPU. Over TCP the admission caps bound the number of streams, but not the number of records per stream: within its thirty-second lifetime a single accepted connection can deliver initiations at the stream's bandwidth, 64 records per read-worker pass. The design document lists "Restore pre-authentication cost defense" as remaining work and notes that a cookie reply would also have to be returned on the exact stream rather than via `wg_socket_send_buffer_as_reply_to_skb ↪ ()`, which would send it over UDP to the synthetic tuple (`kernel/socket.c`, `send4`).

Affected: `kernel/receive.c:265–268` (TCP bypass), `receive.c:272–305` (bypassed cookie, load and MAC logic), `kernel/socket.c` (`wg_socket_send_buffer_as_reply_to_skb` replies over UDP only).

Proof of Concept: Static analysis. Any 148-byte message beginning `01 00 00 00` with random `mac1` and zero `mac2` is consumed in TCP mode; over TCP it is framed as `00 00 00 9c 00 00 b6 25` plus the payload (Appendix A-07). The bypass:

```
/* receive.c:265 */
if(wg->transport == WG_TRANSPORT_TCP) {
   /* For TCP, skip cookie check */
   packet_needs_cookie = false;
   goto nocookie;
}
```

Impact: Pre-authentication CPU exhaustion of the handshake workers. At roughly 50 to 100 microseconds of asymmetric work per 148-byte packet, a few hundred megabits per second of spoofed UDP or a modest

number of accepted TCP streams saturate every CPU, and legitimate handshakes time out. No authentication bypass results; the Noise handshake still fails for forged initiations.

Preconditions: TCP mode; reachability of the UDP companion port (unless firewalled) or of the TCP listener.

Recommendation: Validate `mac1` for every handshake message regardless of transport (initiations already carry it, so this changes no wire format); keep the load computation and the rate limiter, keyed by the stream's remote address for TCP; implement same-stream cookie replies and cookie consumption using the connection id carried in `PACKET_CB(skb)`, as staged in the design document; and offer a TCP-only mode that does not bind the UDP companion socket, so operators do not depend on an external firewall rule.

Classification: CWE-693 · WireGuard protocol (cookie and rate limiting), NIST SP 800-218 PW.4 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N

RVPN-01-606 WP6: Constant cleartext record prefix fingerprints every WGTCP connection (High)

WGTCP frames each WireGuard message with an eight-byte cleartext header: a big-endian 32-bit total length, a type byte that is always 0, a flags byte that is 0 or 1, and a 16-bit "checksum" that is a pure function of those fields (`kernel/socket.h:33–38`, `kernel/wg_tcp.c:280–330`, `2879–2897`). Nothing in the header is secret or randomised, and the WireGuard message that follows begins with its own cleartext type word and three reserved zero bytes. Every WGTCP connection therefore opens with the same bytes: the client's handshake initiation is always 148 bytes, so the first twelve bytes of every stream are `00 00 00 9c 00 00 b6 25 01 00 00 00`, and the server's first reply always begins `00 00 00 64 00 00 a9 25 02 00 00 00`. Keepalives are `00 00 00 28 00 00 a0 a5 04 00 00 00` ↪ . Record lengths expose WireGuard's 16-byte padded packet sizes exactly as UDP does. The listener's behaviour is equally distinctive under active probing: a connection that sends anything else receives silence for five seconds and is then closed, and the companion UDP port silently swallows datagrams.

Affected: `kernel/socket.h:33–38` (`struct wg_tcp_encap_header`), `kernel/wg_tcp.c:280–330` (`wg_tcp_build_frame`), `2879–2897` (`wg_header_checksum`), `3216–3235` (validator).

Proof of Concept: Appendix A-07 executed the framing model and derived the constants:

```
handshake initiation (148 B, type 1) 0000009c0000b625 01000000 -> 12-byte constant prefix 0000009
    ↪ c0000b62501000000
handshake response (92 B, type 2) 000000640000a925 02000000 -> 12-byte constant prefix 000000640000
    ↪ a92502000000
keepalive data (32 B, type 4) 000000280000a0a5 04000000 -> 12-byte constant prefix 000000280000
    ↪ a0a504000000
```

A single static byte-match rule in any deep-packet-inspection engine identifies and blocks the protocol at the first segment.

Impact: A censor can identify every WGTCP user and block the transport nationwide with one rule, which defeats the component's purpose of reaching networks where UDP WireGuard is blocked. Under the engagement's severity rules fingerprintability that lets a censor block users specifically is High unless mitigated.

Preconditions: The WGTCP listener is reachable from the censor's vantage point without an obfuscating layer in front. If production always places the listener behind the obfuscated transport assessed in WP1, WP5 and WP7 and the raw port is not exposed, the residual severity is Low; that topology could not be verified within WP6.

Recommendation: Treat WGTCP as a transport that must never be exposed raw to a hostile network: either front it with the obfuscated carrier, binding the listener to a loopback or private address only, or add a pluggable obfuscation layer to the framing (for example encrypting the record header and the WireGuard type word with a per-connection key derived from the peers' static-static Diffie-Hellman output, adding random-length padding records, and closing the connection immediately on an invalid first record so that the listener behaves like a generic service under probing). Document the exposure assumption in `README.md` and `QUICKSTART.md`.

Classification: CWE-203 · ASVS 5.0.0 V12 (Secure Communication), NIST SP 800-218 PW.1 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N

RVPN-01-607 WP6: Two WQ_MEM_RECLAIM workqueues per peer exhaust kernel threads at scale (High)

In TCP mode `wg_peer_create()` allocates two dedicated workqueues for every configured peer: `alloc_workqueue ↪ ("tcp_read_wq", WQ_UNBOUND | WQ_MEM_RECLAIM, 0)` and `alloc_workqueue("tcp_write_wq", WQ_UNBOUND | ↪ WQ_MEM_RECLAIM, 0)` (`kernel/peer.c:123–137`). The kernel's workqueue implementation creates a dedicated rescuer kernel thread (named `kworker/R-` followed by the workqueue name) for every workqueue flagged `WQ_MEM_RECLAIM`, plus the workqueue structure and its per-node pool workqueues. Each peer therefore costs two kernel threads, two kernel stacks and two workqueue objects, in the order of 50 KB and two process identifiers, before any traffic. The stated goal of the fork is a single interface holding ten million clients (`kernel/messages.h:49–56`, `kernel/peerlookup.h:17–34`), which would require twenty million kernel threads; `kernel. ↪ threads-max` defaults to roughly one thread per 128 KB of RAM (about 500,000 on a 64 GB host) and `pid_max` to 4,194,304. The limit is therefore reached between a few hundred thousand and two million TCP-mode peers. When it is reached, `alloc_workqueue()` fails and `wg_peer_create()` returns `-ENOMEM` gracefully, but the host is already at its thread ceiling: every `fork()` and `clone()` on the system fails with `EAGAIN`, including the provisioning agent and remote administration. The provisional peers created for inbound connections correctly share the per-device `wg->tcp_auth_wq` (`kernel/device.c:92`, `wg_tcp.c:4962`, `4970`), which shows that a shared workqueue satisfies the stream workers' needs. Neither workqueue is on a memory-reclaim path, so `WQ_MEM_RECLAIM` is not justified. All workqueues also carry identical names, which defeats diagnosis. `tools/scale_test.py` never sets `transport tcp`, so the repository's own scaling measurement exercised UDP-mode peers, which allocate no workqueues; the TCP-mode per-peer cost was never measured.

Affected: `kernel/peer.c:123–137` (`wg_peer_create`), `peer.c:216–231` (`peer_make_dead`, per-peer `destroy_workqueue`), `tools/scale_test.py:50–86` (no transport selection).

Proof of Concept: Static analysis:

```
/* peer.c:123 */
if (wg->transport == WG_TRANSPORT_TCP) {
   peer->tcp_read_wq = alloc_workqueue("tcp_read_wq", WQ_UNBOUND | WQ_MEM_RECLAIM, 0);
   ...
   peer->tcp_write_wq = alloc_workqueue("tcp_write_wq", WQ_UNBOUND | WQ_MEM_RECLAIM, 0);
```

Two rescuer threads per peer times the configured maximum of 10,000,000 peers is 20,000,000 kernel threads against a `pid_max` of 4,194,304.

Impact: The ten-million-peer design target is unreachable in TCP mode by a factor of ten or more, and provisioning a TCP-mode interface toward even a few hundred thousand peers exhausts kernel threads and denies fork to every process on the host. The condition is triggered by ordinary, legitimate provisioning.

Preconditions: Administrative provisioning of TCP-mode peers; no attacker involvement.

Recommendation: Use the per-device `tcp_auth_wq` (or a small fixed pool of unbound workqueues selected by peer id) for all peers, remove `WQ_MEM_RECLAIM`, remove the per-peer `destroy_workqueue()` calls, and keep the read and write `work_struct`s per peer as they are. Re-run `tools/scale_test.py` with `transport tcp` and record thread count, PID usage and resident memory per peer at 10,000, 100,000 and 1,000,000 peers before publishing a peer-count claim.

Classification: CWE-770 · NIST SP 800-218 PW.1, Linux workqueue documentation (WQ_MEM_RECLAIM semantics) · CVSS:4.0/AV:L/AC:L/AT:N/PR:H/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:H

RVPN-01-608 WP6: Potential RTNL deadlock in the netns exit path on Linux 7.2 (Medium)

WGTCP registers a FIB notifier in every network namespace (`kernel/device.c:725–739`, `wg_netns_init`) whose callback queues `wn->fib_dispatch_work` with zero delay on every route, rule or nexthop event (`device.c:243–261`). The worker, `wg_tcp_fib_dispatch_worker()`, begins with `rtnl_lock()` to walk `device_list` (`device.c:224–240`). On kernels 7.2 and later the namespace exit hook is the `.exit_rtnl` pernet operation (`device.c:746`, `805`), which the core invokes with RTNL already held (the `dev_kill_list` parameter exists so that callees collect devices for a single `unregister_netdevice_many()` under that lock). Inside that hook WGTCP calls `cancel_delayed_work_sync(& ↪ wn->fib_dispatch_work)` (`device.c:761`). If the dispatch worker is executing at that moment, it is blocked in

`rtnl_lock()` waiting for the lock the exiting context holds, while the exiting context waits for the worker to finish: a classic lock-order inversion. The pre-7.2 variant cancels the work before taking RTNL itself (`device.c:763–764` ↪) and is not affected. The window opens whenever a FIB change in a namespace is followed closely by that namespace's teardown, which is the normal sequence when a container or test namespace is torn down after its routes are removed; it is not remotely triggerable. Because the hanging task is the `cleanup_net` worker holding RTNL, every subsequent netlink configuration call on the host blocks.

Affected: `kernel/device.c:746–764` (`wg_netns_exit_rtnl`, `cancel_delayed_work_sync` under RTNL), `device.c:224–240` (`wg_tcp_fib_dispatch_worker`, `rtnl_lock`), `device.c:725–739` (notifier registered in every netns).

Proof of Concept: Static analysis; lock order:

```
cleanup_net (RTNL held) -> wg_netns_exit_rtnl -> cancel_delayed_work_sync(fib_dispatch_work) -> waits for
    ↪ worker
worker: wg_tcp_fib_dispatch_worker -> rtnl_lock() -> waits for cleanup_net
```

The condition is potential: it was derived from the code and the documented `.exit_rtnl` contract and was not reproduced, because no Linux kernel is available on the assessment host. The missing step is a 7.2 host running `ip netns add t; ip -n t route add 192.0.2.0/24 dev lo; ip netns del t` in a loop with the module loaded.

Impact: Host-wide hang of network configuration (RTNL held forever) and a stuck `cleanup_net`; recovery requires a reboot. Affects fleet hosts on 7.2 that create and destroy namespaces, such as container hosts and CI runners.

Preconditions: Linux 7.2 or later, module loaded, a namespace teardown shortly after a FIB change in that namespace.

Recommendation: Do not wait for the worker under RTNL: clear `fib_registered`, call `cancel_delayed_work()` ↪ (non-sync) in the hook, and have the worker re-check `fib_registered` under `rtnl_lock()` and return, or use `rtnl_trylock()` with requeue in the worker. Register the FIB notifier only in namespaces that contain a TCP-mode WGTCP device instead of in every namespace, which also removes the per-namespace route dump at registration.

Classification: CWE-833 · Linux locking documentation (lock ordering, work cancellation), NIST SP 800-218 PW.5 · CVSS:4.0/AV:L/AC:H/AT:P/PR:H/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:H

Miscellaneous Issues

RVPN-01-609 WP6: First handshake response on a new carrier is dropped before promotion (Medium)

When an initiation arrives on a provisional (accepted) stream, `wg_receive_handshake_packet()` first calls `wg_socket_set_peer_endpoint_authenticated_from_skb()` (`kernel/receive.c:330`), which marks the carrier authenticated and queues promotion work on `system_wq` (`kernel/wg_tcp.c:648–660`), and then immediately calls `wg_packet_send_handshake_response(peer)` (`receive.c:332`). The response is sent through `wg_socket_send_skb_to_peer()` (`wg_tcp.c:410–476`), which in TCP mode requires `peer->peer_socket` to be set and established. For a configured peer without a current carrier that pointer is `NULL` until the promotion worker has completed `synchronize_rcu()` (`wg_tcp.c:4446`), taken two mutexes and transferred the socket (`wg_tcp.c:4536–4538`), which takes milliseconds, whereas the response is built and sent microseconds after the work is queued; the `else` branch therefore frees the response (`wg_tcp.c:446–461`). For a roaming reconnect whose previous carrier is dead, the response is instead queued to that dead carrier and purged at promotion (`wg_tcp.c:4509–4522`, `1105–1108`). Nothing sends anything on the new carrier after promotion. The client recovers only by the protocol's handshake retransmission after `REKEY_TIMEOUT` (5 s), so every TCP carrier establishment costs at least five seconds before the tunnel passes traffic. The condition is potential: it is deterministic from the code's ordering but was not timed, because the module cannot run here. The missing step is a two-namespace Linux run comparing the TCP connect time with the `latest-handshake` time on a fresh carrier.

Affected: `kernel/receive.c:328–332`, `kernel/wg_tcp.c:410–476` (`wg_socket_send_skb_to_peer`), `648–660` (promotion queued asynchronously), `4406–4605` (`wg_tcp_promote_authenticated_carrier`).

Proof of Concept: Static analysis (ordering shown above); no dynamic measurement.

Impact: A five-second connect delay on every carrier establishment and reconnect, which users of a mobile VPN experience as a hung "connecting" state and which the client library may be masking with aggressive retransmission.

Preconditions: TCP mode; any client connecting or reconnecting.

Recommendation: Reply to pre-session messages on the exact stream that carried them: carry the connection id into `wg_packet_send_handshake_response()` and enqueue the response onto that stream's send queue (the provisional peer's) when the configured peer has no carrier yet; alternatively perform promotion synchronously before sending the response by replacing the entry's `synchronize_rcu()` with `call_rcu()`, and send a keepalive from the server immediately after promotion.

Classification: CWE-362 · WireGuard protocol (handshake retransmission timers), NIST SP 800-218 PW.8 (testing) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N

RVPN-01-610 WP6: Inner IPv4 fragmentation metadata sent in cleartext (Low)

Before encryption, `wg_packet_send_staged_packets()` extracts the IPv4 identification and fragment-offset fields of the inner packet (`kernel/send.c:335–351`, `547–560`) and stores them in the packet control block. `wg_tcp_build_frame ↪ ()` then emits them in a four-byte cleartext fragment header behind the record header whenever `frag_off` is non-zero (`kernel/wg_tcp.c:288–300`, `321–323`). On receive the values are written into the synthetic outer IP header (`wg_tcp.c:3804–3812`) and are otherwise unused; the inner packet still carries its own fragmentation fields inside the AEAD envelope, so the mechanism serves no function. The effect is a leak: an observer learns, in cleartext, that the inner traffic contains IPv4 fragments and sees the inner IP identification sequence, which supports host fingerprinting and flow correlation, and the extra four bytes make fragmented records distinguishable by length.

Affected: `kernel/send.c:335–351`, `547–560`; `kernel/wg_tcp.c:288–300`, `321–323`, `3804–3812`; `kernel/socket.h:39–42`.

Proof of Concept: Static analysis; the fragment flag and header are produced only for inner IPv4 packets with `IP_MF` or a non-zero offset.

Impact: Confidentiality leak of inner-packet metadata and a secondary fingerprint; low because inner fragmentation is rare at the default MTU.

Preconditions: Inner IPv4 fragments traversing the tunnel.

Recommendation: Remove the fragment header and the `frag_id`/`frag_off` control-block fields entirely; the WireGuard payload is self-describing and the validator becomes simpler.

Classification: CWE-319 · ASVS 5.0.0 V12 (Secure Communication) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/S

RVPN-01-611 WP6: Unratelimited kernel logging on remotely triggerable paths (Low)

Fix Note: This issue has been fixed by the development team in commit c7c0260: the accept-error, address-read and state-change sites are ratelimited (`kernel/wg_tcp.c:1267`, `1293`, `2232`), the receive-error site is a `net_dbg_ratelimited()` (`wg_tcp.c:3246`), the pointer is printed with `%p`, and the per-packet warning in `print_peer_socket_info()` is now a `wg_dbg()` (`kernel/wg_tcp_debug.c:942`); the helper is still called on the transmit and receive paths but compiles to a no-op in production builds. The fix was verified by Red Team Ltd by code review of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

`kernel/wg_tcp.c` contains 61 `pr_err`, `pr_warn`, `printk(KERN_ERR)` and `printk(KERN_WARNING)` sites against five ratelimited ones. Several are driven by the remote side: `wg_tcp.c:3551` logs every `kernel_recvmsg()` error with the peer id, which fires for every connection the remote resets (`ECONNRESET` is the normal end of a mobile client's carrier); `wg_tcp.c:1404` logs every accept error other than the handled set; `wg_tcp.c:1437` logs address-read failures; `wg_tcp.c:2434` logs a raw kernel pointer with `%px` in a production `pr_err`; and `kernel/wg_tcp_debug.c:942` emits a `KERN_WARNING` from `print_peer_socket_info()`, which is called on every transmitted packet (`wg_tcp.c:427`) and on every read-worker pass (`wg_tcp.c:3479`) even in production builds, where the surrounding `wg_dbg()` calls are compiled out but the function call and its comparisons are not. An attacker, or merely a large fleet of clients, can therefore generate unbounded `dmesg` and journal volume, and the `%px` site would print an unhashed pointer if ever reached.

Affected: kernel/wg_tcp.c:1404, 1437, 2434, 3551; kernel/wg_tcp_debug.c:906–944 (print_peer_socket_info); callers at wg_tcp.c:427, 559, 1124, 1573, 2089, 2452, 2792, 3479.

Proof of Concept: Appendix A-13 inventory: 61 unratelimited log sites, 5 ratelimited, one %px outside wg_dbg.

Impact: Log flooding and console serialisation under adversarial connection churn; an unhashed kernel pointer in logs weakens KASLR if the path is hit.

Preconditions: Network reachability; for the %px site, an internal state inconsistency.

Recommendation: Convert remote-driven sites to net_dbg_ratelimited() or pr_debug_ratelimited(), print pointers with %p or not at all, and remove print_peer_socket_info() from the transmit and receive hot paths (make it a wg_dbg-only macro).

Classification: CWE-779 · ASVS 5.0.0 V16 (Security Logging and Error Handling), Linux printk documentation · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N

RVPN-01-612 WP6: Diagnostic builds log key material, plaintext and kernel pointers (Low)

Fix Note: This issue has been fixed by the development team in commit cc6ea2f: the encryption and decryption key dumps, the plaintext dumps before encryption and after decryption, the staged-packet dump and the netlink private-key and preshared-key dumps are removed (kernel/netlink.c:846 and 903 now print lengths only), and every %px in the module is replaced by %p (no occurrence remains). Residual verbose output is limited to handshake messages and the ciphertext of packets that failed decryption, neither of which is secret; the build-time guard and MODULE_INFO marker recommended below were not added. The fix was verified by Red Team Ltd by code review of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

With -DWG_TCP_VERBOSE the module logs the session encryption key (kernel/send.c:310), the decryption key (kernel/receive.c:492), packet plaintext before encryption and after decryption (send.c:258, receive.c:517), complete handshake messages (receive.c:263), and, under the inline netlink's DIAGNOSTIC option, the device private key and peer preshared keys from netlink attributes (kernel/netlink.c:903 and 846). More than one hundred wg_dbg() sites print raw kernel addresses with %px. WG_TCP_DIAG output is unratelimited by design. These levels are off unless added to ccflags, and the DKMS build passes none of them; but kernel/Makefile:28 carries the enabling line as a comment one edit away, Makefile:7 turns CONFIG_WIREGUARD_DEBUG from the host kernel configuration into -DDEBUG (which enables the self-tests and, together with WG_TCP_FAULT_INJECTION, the destructive fault parameters), and nothing in the module metadata distinguishes a verbose build from a production one. The design document itself states that "WG_TCP_VERBOSE can expose secrets".

Affected: kernel/send.c:258, 310; kernel/receive.c:263, 492, 517; kernel/netlink.c:846–903; kernel/Makefile:7, 28; kernel/wg_tcp_debug.h.

Proof of Concept: Appendix A-13 grep inventory of key-printing wg_dbg sites.

Impact: If a verbose build reaches a production host, session and static keys and user plaintext are written to the kernel log, readable by any process with dmesg access and shipped to central logging.

Preconditions: A verbose or diagnostic build deployed outside an isolated lab.

Recommendation: Delete the key- and plaintext-printing statements rather than gating them; add #if defined (WG_TCP_VERBOSE)&& !defined(DEBUG)#error "verbose builds require DEBUG" #endif; replace %px with %p; add MODULE_INFO(wgtcp_diag, "verbose") so that modinfo reveals diagnostic builds; and have the DKMS build reject unexpected ccflags.

Classification: CWE-532 · ASVS 5.0.0 V14 (Data Protection), NIST SP 800-218 PW.6 · CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/V

RVPN-01-613 WP6: TCP listeners created with SO_REUSEPORT (Low)

Fix Note: This issue has been fixed by the development team in commit 6a7bdf1: both listeners call sock_set_reuseaddr() before binding (kernel/wg_tcp.c:1520, 1582) and sock_set_reuseport() is gone, which also removes the inconsistency between the address families. The fix was verified by Red Team Ltd by code review

of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

The IPv4 kernel listener calls `sock_set_reuseport()` before binding (`kernel/wg_tcp.c:1700`); the IPv6 listener (`1727–1789`) does not, so the two address families also behave inconsistently. Kernel sockets carry the root user id, so any process running as root in the same network namespace can bind the same TCP port with `SO_REUSEPORT` and join the reuseport group, after which the kernel distributes incoming WGTCP connections between the module and that process. The practical effects are silent loss of a share of inbound carriers to an unrelated service that happens to use the same port with reuseport, and a local interposition primitive for a compromised root process. `SO_REUSEPORT` is not needed to re-bind a listening port after a restart; `SO_REUSEADDR` or nothing suffices for a listener.

Affected: `kernel/wg_tcp.c:1700` (`wg_setup_tcp_listen4`); `1727–1789` (`wg_setup_tcp_listen6`, no reuseport, inconsistent with IPv4).

Proof of Concept: Static analysis: `sock_set_reuseport(socket->sk);` precedes `kernel_bind()`.

Impact: Connection stealing by a co-resident root process or port collision with another reuseport service; low because root is already privileged.

Preconditions: A root process binding the same port with reuseport in the device's creating namespace.

Recommendation: Remove `sock_set_reuseport()`; use `sock_set_reuseaddr()` if re-binding during `TIME_WAIT` is a concern, and fail interface activation if the port is in use.

Classification: CWE-668 · Linux netdev and socket documentation, NIST SP 800-218 PW.5 · CVSS:4.0/AV:L/AC:L/AT:N/PR:H/

RVPN-01-614 WP6: Debian packaging installs into /usr/local and departs from Policy (Low)

The `wgtcp-dkms` package compiles the `wg` fork in `postinst` and installs it as `/usr/local/bin/wg` (`debian/postinst:38–44`), and `prerm` removes `/usr/local/bin/wg` unconditionally on package removal (`debian/prerm:18–22`). Debian Policy 9.1.2 reserves `/usr/local` for the system administrator and forbids packages from installing files there; the file is not tracked by dpkg, and the `prerm` deletes whatever the administrator has placed at that path, for example a locally built upstream `wg`. The package `Depends` on `build-essential` (`debian/control:11–15`), which Policy 4.2 says packages must not depend on; `Standards-Version: 4.6.2` (`control:6`) predates the current 4.7 series; the maintainer scripts are hand-written rather than generated by `dh-sequence-dkms`, so upgrade ordering and the `dkms` hooks are not the audited defaults; `debian/dkms.conf.in` lacks `BUILD_EXCLUSIVE_KERNEL`, so DKMS attempts a build on kernels outside the declared 6.12 to 7.2 range; `debian/control` and `debian/changelog` name different maintainers; and the built module is not signed, so Secure Boot hosts cannot load it. Because the module is installed as `wireguard.ko` under `updates/dkms`, it replaces the in-tree WireGuard for every interface on the host, including UDP-only ones, which then pay the 96 MiB per-interface hashtable allocation (`kernel/peerlookup.h:17–37`) and inherit the raised peer limit.

Affected: `debian/postinst:38–44`, `debian/prerm:18–22`, `debian/control:4–15`, `debian/dkms.conf.in:1–8`, `debian/rules:17–33`.

Proof of Concept: Appendix A-03 (scripts syntax-check clean) and A-06 (`rules` parses); the Policy deviations are visible in the quoted lines.

Impact: Administrator-owned files are overwritten and deleted by package operations; the package fails Policy and lintian checks; unsupported kernels receive failed builds; the fleet's non-WGTCP WireGuard interfaces run the fork.

Preconditions: Package installation or removal on a fleet host.

Recommendation: Build `wg` at package build time into an architecture-specific binary package as `/usr/bin/wgtcp` (or install the fork via `update-alternatives` against `wireguard-tools`), use `dh-sequence-dkms` with a generated `dkms.conf` containing `BUILD_EXCLUSIVE_KERNEL="^(6\.12|7\.[0-2])\."`, replace `build-essential` with `gcc | c-compiler, make, libmnl-dev, pkgconf`, update `Standards-Version` to 4.7.2, align the maintainer fields, sign the module with a fleet MOK, and consider naming the module `wgtcp` with its own `rtnl_link` kind so that stock interfaces keep the in-tree module.

Classification: CWE-710 · Debian Policy 4.7.2 §4.2 and §9.1.2, DKMS conventions, NIST SP 800-218 PS.1 · CVSS:4.0/AV:L/AC:L/AT:N/PR:H/UI:N/VC:N/VI:L/VA:L/SC:N/SI:N/SA:N

RVPN-01-615 WP6: WQ_PERCPU header probe misses Debian flavour and Ubuntu header layouts (Low)

`kernel/Makefile:18–21` detects `WQ_PERCPU` by grepping `include/linux/workqueue.h` in the kernel build directory and in a sibling derived by `sed 's/-[a-z0-9_]*$/-common/'`. The derivation assumes the Debian naming `linux-headers-VERSION-ARCH` with a `-common` sibling. Appendix A-05 executed the expression on representative paths: it is correct for `-amd64` and `-arm64`, but maps `...-cloud-amd64` to `...-cloud-common` (the real sibling is `...-common`), `...-rt-amd64` to `...-rt-common` (the real sibling is `...-common-rt`), and Ubuntu's `linux-headers-6.14.0-29-generic` to `...-29-common` (Ubuntu uses `linux-headers-6.14.0-29` with no suffix). On those kernels the probe silently yields no match, `HAVE_WQ_PERCPU` is not defined and `WG_WQ_PERCPU` expands to 0 (`kernel/device.c:31–35`), so the handshake and crypto workqueues are allocated without `WQ_PERCPU` on kernels that have the flag. Today the kernel still treats the absence of both `WQ_PERCPU` and `WQ_UNBOUND` as per-CPU, so the build succeeds and behaves as intended, but the probe is a silent misdetection that will change behaviour or emit warnings the moment the kernel enforces the flag, and it already departs from the upstream 7.2 baseline, which passes `WQ_PERCPU` explicitly.

Affected: `kernel/Makefile:18–21`, `kernel/device.c:27–35`, `635–648`.

Proof of Concept: Appendix A-05:

```
/usr/src/linux-headers-6.12.43+deb13-cloud-amd64 -> /usr/src/linux-headers-6.12.43+deb13-cloud-common
/usr/src/linux-headers-6.12.43+deb13-rt-amd64 -> /usr/src/linux-headers-6.12.43+deb13-rt-common
/usr/src/linux-headers-6.14.0-29-generic -> /usr/src/linux-headers-6.14.0-29-common
```

Impact: Silent loss of the intended workqueue affinity on cloud, real-time and Ubuntu kernels, and future build or runtime surprises when the flag becomes mandatory.

Preconditions: Building on a Debian cloud or rt flavour or on Ubuntu.

Recommendation: Replace the path heuristic with a compile test against the kernel's include path, in the style of kbuild's `try-run` (compile a one-line file that references `WQ_PERCPU` with `$(CC)$(KBUILD_CFLAGS)` and define `HAVE_WQ_PERCPU` on success), or probe `$(objtree)/include/generated/autoconf.h` together with the resolved `-common` directory read from the `Makefile` symlink target rather than from a regex.

Classification: CWE-440 · NIST SP 800-218 PW.6 (build toolchain configuration) · CVSS:4.0/AV:L/AC:H/AT:P/PR:H/UI:N/VC:N

RVPN-01-616 WP6: UAPI attribute reuse and configuration portability of the wg fork (Info)

Fix Note: This issue has been partially fixed by the development team in commit 7eab4e2. Fixed part: `showconf` emits `Transport = tcp` only for TCP interfaces (`tools/showconf.c:49–50`), so saved configurations of UDP interfaces load on stock tools again; Red Team Ltd rebuilt the tool natively and re-ran the sanitiser harness on the remediated sources (Appendix A-20). Open parts: the attribute number, the hand-edited generated policy and the dump-based capability check in `kernel_set_device()` are unchanged. The partial fix was verified by Red Team Ltd by code review of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

Four observations on the control-plane ABI and the tool. First, the shim defines `WGDEVICE_A_TRANSPORT` as the stock enum's `__WGDEVICE_A_LAST` sentinel (value 9, `kernel/wireguard_tcp_uapi.h:36–40`) and the tool's header appends it at the same value (`tools/uapi/linux/linux/wireguard.h:170`); Appendix A-09 confirmed the two agree and that both netlink policies declare it as `NLA_U8`. The number is not coordinated with upstream, so the next upstream device attribute will collide: a newer stock `wg` would send attribute 9 with another meaning to a WGTCP kernel, and the WGTCP `wg` would send transport 0 or 1 to a newer stock kernel as whatever attribute 9 then means. Second, `kernel/generated/netlink.c` is hand-edited despite its "Do not edit directly" header and no `wireguard.yaml` specification is updated, so YNL-based consumers and any regeneration lose the attribute. Third, `showconf` now prints `Transport = udp` for every interface on a WGTCP kernel (`tools/showconf.c:49–50`), so `wg-quick` with `SaveConfig` writes a line that stock `wg` rejects as "Line unrecognized" (Appendix A-11 shows the fork itself rejecting the key outside `[Interface]`). Fourth, `kernel_set_device()` performs a complete `WG_CMD_GET_DEVICE` dump of all peers whenever `transport` is specified (`tools/ipc-linux.h:154–167`), which on the intended ten-million-peer interface is a multi-gigabyte netlink dump for a one-byte attribute. On the positive side, the fork

fails closed: a `transport tcp` request against a kernel that does not report the attribute, or against a userspace implementation, returns `EOPNOTSUPP` (A-11), and the `EBUSY` semantics for live changes are documented in `wg.8`.

Affected: `kernel/wireguard_tcp_uapi.h:36-40`, `kernel/generated/netlink.c:46-55`, `tools/uapi/linux/linux/wireguard.h:146-172`, `tools/showconf.c:49-50`, `tools/ipc-linux.h:154-167`.

Proof of Concept: Appendix A-09 and A-11 outputs.

Impact: Forward ABI collision risk; configuration files that stop loading on stock tools; an expensive control-plane operation.

Preconditions: Future upstream attribute additions; mixed stock and fork tooling.

Recommendation: Bump `WG_GENL_VERSION` in the fork and have the tool probe the family version before sending the attribute; propose the attribute upstream or move it into a nested vendor attribute; regenerate `generated/netlink.c` from an updated YAML; emit `Transport` from `showconf` only when the value is `tcp`; and replace the full dump with a cheap capability check (a `GET_DEVICE` with a peer-less interface is still a dump, so cache the capability per interface name).

Classification: CWE-1059 · NIST SP 800-218 PW.1, Linux netlink documentation (YAML specifications) · CVSS:4.0/AV:L/AC:L/AT:N/PR:H/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-617 WP6: Dead code, protocol deviation and portability defects in the kernel delta (Info)

Fix Note: This issue has been partially fixed by the development team in commits e0cd9e4, 54d9b09 and 1772c2c. Fixed parts: the dead helpers, declarations and write-only fields are removed (`wg_tcp_queuepkt`, `wg_find_peer_by_endpoints`, `wg_clean_peer_socket`, `wg_release_peer_tcp_connection`, `wg_free_peer_socket_data`, the `wg_tcp_connection_retry_timer` prototype, `tcp_cleanup_lock`, `pending_connection_list`, `clean_inbound` and `clean_outbound`); the outbound IPv6 dial path uses `IS_ENABLED(CONFIG_IPV6)` (`kernel/wg_tcp.c:1978, 2095`); the unreachable check that leaked an skb is removed; the two macros are parenthesised (`kernel/socket.h:59-60`); and the listener socket pointers no longer carry a `__rcu` annotation (`kernel/device.h:83`). Open parts: the `noise.c` tie-break, the MTU and headroom accounting, the duplicated pre-7.0 netlink implementation, the ignored `copy_sock_addresses()` return value, and four debug-only `rcu_access_pointer()` calls on the now plain listener pointers (`kernel/wg_tcp_debug.c:848-864`), which sparse would flag. The partial fix was verified by Red Team Ltd by code review of the isolated remediation delta (Appendix A-16) and by the contract suite (Appendix A-17); the change has not been compiled or loaded.

Code quality items that do not have a direct exploit path but should be fixed before the module is treated as production software. Dead code: `wg_tcp_queuepkt()` (`kernel/wg_tcp.c:2775-2876`) has no caller; `wg_find_peer_by_endpoints()` (`1351-1381`) is reachable only from it; `wg_clean_peer_socket()` (`1117-1198`) has no caller and performs lock-free socket teardown that would be unsafe if ever used; `wg_release_peer_tcp_connection()` is marked `__maybe_unused` (`2344`); `wg_free_peer_socket_data()` (`1045`) is unused; `wg_tcp_connection_retry_timer()` is declared (`kernel/socket.h:80`) but never defined; `tcp_cleanup_lock` (`kernel/device.h`) is initialised and never used; `pending_connection_list`, `clean_inbound` and `clean_outbound` in `struct wg_peer` are written and never read. Protocol deviation: `kernel/noise.c:664-679` discards a valid handshake initiation when the local static public key compares lower and local state is `HANDSHAKE_CREATED_INITIATION`, for TCP devices only; the WireGuard protocol has no such tie-break (crossing initiations are resolved by the state machine and timers), the branch runs after the expensive consumption, and it changes convergence timing between peers that initiate simultaneously. MTU accounting: `wg_setup()` evaluates `wg->transport` before netlink can set it, so the TCP overhead term at `kernel/device.c:555-565` is always zero and the default MTU ignores the 20-byte TCP header and the 8-byte record header; the design document lists this as open. Portability: `#ifdef CONFIG_IPV6` at `wg_tcp.c:2158` and `2275` compiles the outbound IPv6 dial path out on kernels with `CONFIG_IPV6=m`, while every other site uses `IS_ENABLED()`. Latent leak: the unreachable check at `kernel/receive.c:653-657` returns without freeing the skb. Macros: `WG_MAX_PACKET_SIZE` and `WG_TCP_SKB_READ_ALLOC_SIZE` lack parentheses (`kernel/socket.h:59-60`). Annotations: `tcp_listen_socket4/6` are declared `__rcu` but accessed without RCU accessors (`kernel/device.h`). Duplication: the pre-7.0 netlink implementation duplicates roughly 1,000 upstream lines (`kernel/netlink.c:640-1674`). The ignored return value of `copy_sock_addresses()` (`wg_tcp.c:1548`) leaves `inbound_source` zeroed on failure, which the fake-header builder then rejects correctly. Style: mixed indentation, declarations after statements and bare `printk()` without `pr_fmt` would be flagged by checkpatch, which could not be run here.

Affected: As cited above across `kernel/wg_tcp.c`, `noise.c`, `device.c`, `receive.c`, `socket.h`, `device.h`, `peer.h`, `netlink.c`.

Proof of Concept: Appendix A-13 caller inventory (zero call sites for `wg_tcp_queuepkt`, `wg_clean_peer_socket`).

Impact: Maintenance risk and latent bugs; no direct security impact.

Preconditions: None.

Recommendation: Remove dead code; drop the `noise.c` tie-break or document it as a deliberate deviation and move it before the DH work; recompute `needed_headroom` and the default MTU when the transport is set; use `IS_ENABLED(CONFIG_IPV6)` consistently; run sparse, checkpatch and smatch in CI.

Classification: CWE-561 · Linux kernel coding style, NIST SP 800-218 PW.7 (code review) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/U

RVPN-01-618 WP6: Carrier liveness and displacement behaviour (Low)

Two carrier lifecycle behaviours deserve attention. First, no carrier sets `TCP_USER_TIMEOUT` or keepalives: accepted sockets receive only `TCP_NODELAY` and the device mark (`kernel/wg_tcp.c:1413–1418`), and dialled sockets only the mark and a 900-second blocking timeout that nonblocking I/O never uses (`wg_tcp.c:2199–2246`, `766–782`). A half-open carrier (a NAT mapping that expired without a reset) is detected only by the TCP stack's retransmission limit, by default about fifteen minutes, during which the write worker sees `EAGAIN` and the per-peer queue fills to `MAX_QUEUED_PACKETS` frames (`wg_tcp.c:364–408`), up to tens of megabytes per peer; the design document lists stall detection as a required control. Second, promotion is driven by any authenticated packet on a newer provisional stream and actively shuts the previous carrier (`wg_tcp.c:4499–4522`). WireGuard's UDP roaming has the same "last authenticated packet wins" rule, under which an adversary who captures a packet and delivers a copy from another source before the original moves the endpoint; the replay counter restricts this to packets the server has not yet seen. In WGTCP a successful race through a fresh TCP connection (which carries a newer connection id, `wg_tcp.c:4422`, `4460`) additionally tears down the victim's live stream, turning a transient misdirection into a forced reconnect. An adversary positioned to win that race can already drop the victim's traffic, so the added impact is small.

Affected: `kernel/wg_tcp.c:1413–1418`, `2199–2246`, `766–782` (no user timeout or keepalive); `4499–4522` (displaced carrier shut down).

Proof of Concept: Static analysis.

Impact: Slow detection of dead carriers with bounded but large memory held per peer; harder reconnect behaviour under replay-driven displacement.

Preconditions: NAT timeouts without reset; for displacement, an on-path or tap-and-inject adversary.

Recommendation: Set `TCP_USER_TIMEOUT` (30 to 45 s) and `SO_KEEPALIVE` on every carrier and add a forward-progress stall detector; keep a displaced carrier open for a short grace period and prefer handshake-driven promotion over data-driven promotion where the client library performs a handshake on every new connection.

Classification: CWE-404 · RFC 9293 (TCP user timeout), WireGuard protocol (roaming) · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N

RVPN-01-619 WP6: Personal mailboxes and lab keys in the public tree (Info)

Fix Note: The maintainer identity in `debian/control` has been replaced by the development team in commit bdbfb96 with the project's public identity, matching `debian/changelog`, and verified by Red Team Ltd to be working as expected. The remaining items listed below (the project's own public contact address, the upstream author attributions in the investigation notes, the forge no-reply address, the lab virtual-machine public keys with their RFC 1918 topology, and the two addresses inherited from the upstream manual page) remain in the tree by design as an accepted risk.

The publication policy for this engagement excludes personal mailboxes and cryptographic keys from the public trees. The scan in Appendix A-02 found the following classes of value (values are deliberately not reproduced): a personal mailbox in a copyright line at `tools/scale_test.py:2`; the same personal mailbox in `debian/changelog:9`, `19` and `29`; a third-party maintainer mailbox at `debian/control:4`; a personal mailbox at `perf-test/meltdown/INVESTIGATION_STATUS.md:260` and `263`; a forge no-reply mailbox at `perf-test/harness/refresh-report.ps1:95`;

two WireGuard public keys of lab virtual machines at `docs/TUNNEL_CONFIG.sh:28`, `32`, `42`, `43`, `65`, `81`, `82`, `104` and `docs/NODE_README.md:81`, `93`, together with an RFC 1918 lab topology (the private keys are stated to be provisioned separately and are not present). The upstream manual page `tools/man/wg.8:195` and `205` carries two public addresses outside the documentation ranges; they are inherited unchanged from wireguard-tools and are not RostamVPN infrastructure. No OAuth client identifiers, private keys, CDN hostnames or absolute user paths were found.

Affected: Files and lines as listed.

Proof of Concept: Appendix A-02.

Impact: Exposure of personal contact data and lab identifiers; no operational secret is exposed.

Preconditions: None.

Recommendation: Replace personal mailboxes with a project role mailbox or remove them; regenerate the lab key pairs and replace the documented public keys with the upstream example keys; optionally patch the inherited manual page examples to RFC 5737 and RFC 3849 addresses.

Classification: CWE-359 · Engagement redaction policy, EU Regulation 2024/2847 Annex I (data minimisation) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-620 WP6: Live listen-port change in TCP mode is not rejected on Linux 7.0 and later (Low)

Found during the version 1.1 re-verification through a pre-existing failure of `tests/test_tcp_port_contract.py` (Appendix A-17); the condition is present in the acd17ec tree and is not introduced by the remediation. The module carries two netlink implementations. The pre-7.0 inline implementation refuses a `listen-port` change on a running TCP-mode device with `EBUSY` (`kernel/netlink.c:1281–1285`), which is the behaviour the fork's manual page documents: "A TCP listen port may only be changed while the interface is down; a live change fails with EBUSY and leaves the existing listeners running" (`tools/man/wg.8:66`). The 7.0+ implementation's `set_port()` (`kernel/netlink.c:295–308`) is the unchanged upstream function. On a running TCP-mode device it re-creates the UDP sockets on the new port through `wg_socket_init()` and updates `wg->incoming_port`, while the TCP listeners that `wg_open()` created keep the old port (`kernel/wg_tcp.c:1472–1600`) and existing carriers continue unchanged. From that moment `wg show` and `showconf` report a listen port on which no TCP listener exists, a configuration saved by `wg-quick` records the wrong port, and the UDP and TCP carriers of one device listen on different ports until the next down and up cycle. The contract test that encodes the `EBUSY` rule searches the first `set_port()` definition in the file, which is the 7.0+ branch, and fails for exactly this reason; it has been failing since the 7.0+ branch was added.

Affected: `kernel/netlink.c:295–308` (7.0+ `set_port`, no guard), `1281–1285` (pre-7.0 guard), `tools/man/wg.8:66` (documented contract).

Proof of Concept: Static analysis plus the contract output in Appendix A-17:

```
FAIL: test_live_tcp_port_change_is_rejected_before_mutation (tests.test_tcp_port_contract.TcpPortContract
    ↪ ...)
AssertionError: 'if (wg->transport == WG_TRANSPORT_TCP && netif_running(wg->dev))\n\t\treturn -EBUSY;'
not found in 'static int set_port(struct wg_device *wg, u16 port)\n{ ... return wg_socket_init(wg, port)
    ↪ ;\n}'
```

Impact: An administrative action that the documentation says is refused instead leaves the device silently inconsistent on the fleet's 7.x kernels, with misleading `wg show` output and wrong saved configurations; a provisioning system that moves ports live will believe the TCP listener moved. Low: it requires `CAP_NET_ADMIN` and has no remote trigger.

Preconditions: Linux 7.0 or later, a running TCP-mode device, `wg set IFACE listen-port PORT`.

Recommendation: Hoist the guard into a helper shared by both implementations so that it cannot diverge again:

```
static int wg_set_port_checked(struct wg_device *wg, u16 port)
{
   if (wg->incoming_port == port)
```

```
        return 0;
    if (wg->transport == WG_TRANSPORT_TCP && netif_running(wg->dev))
        return -EBUSY;
    return set_port(wg, port);
}
```

Make the contract suite a gating check in CI so that a failing contract blocks a release, and fix the four text-drift contracts listed in Appendix A-17 so that genuine failures stand out.

Classification: CWE-1068 · NIST SP 800-218 PW.8 (testing), Linux netdev documentation · CVSS:4.0/AV:L/AC:L/AT:N/PR:H/U

Conclusions

Red Team Ltd reviewed the WGTCP kernel module, its `wg` tool fork and its Debian packaging over two days in October 2026 with the complete delta against upstream WireGuard isolated and read in full. The cryptographic core is untouched upstream code, the carrier ownership model was built with evident care, and the maintainers' own design document is candid about what remains open. The weaknesses the review found are concentrated in three places: the handling of connections before authentication, the assumptions behind the ten-million-peer target, and the operational surroundings of the module (logging, packaging, build probes). The areas of the work package are taken in turn, and each records the state after the maintainers' remediation pass.

Remediation state at 97c267e. Eleven commits on top of bdbfb96 were re-verified by inspection against a reconstructed pre-remediation tree and by the contract suite (Appendix A-16 to A-20); none has been compiled or loaded. They remove the per-accept peer-list walk and the dead helpers (RVPN-01-602, part of 617), consume cookie replies first and validate `mac1` on TCP handshakes while replacing the assertion with a ratelimited drop (RVPN-01-601 fixed, 605 partially fixed), replace `SO_REUSEPORT` with `SO_REUSEADDR` on both listeners (613), ratelimit the remote-driven log sites (611), remove the key, plaintext and raw-pointer output from verbose builds (612), restrict `showconf` to emitting `Transport = tcp` (part of 616), fix the IPv6 build guard, the unreachable leak and the macro hygiene (part of 617), and regenerate the patch, which reproduces the current tree exactly (A-18). The items that define the module's exposure at scale, RVPN-01-603, 604, 606 and 607, and the Medium items 608 and 609, are unchanged, and the re-verification added RVPN-01-620.

Listener and accept path. One kernel thread per address family blocks in `kernel_accept()` on a wildcard socket bound to the port selected by the companion UDP socket (`kernel/wg_tcp.c:1384–1619`, `1672–1904`). Shutdown ordering is correct and is pinned by the contract tests in `tests/test_tcp_listener_contract.py`. The accept thread does far too much per connection, however: the functionless peer-list walk of RVPN-01-602 and the synchronous construction and, on rejection, destruction of a temporary peer with callbacks, module reference and work cancellation all run on it. The recommendation is to reduce the loop to accept, address read, cheap throttles and a hand-off to a device workqueue. Commit e0cd9e4 removed the walk and the dead helpers (RVPN-01-602 fixed); the synchronous construction and destruction of temporary peers on the accept thread remains.

Unauthenticated connections before the first handshake. The provisional-peer model is sound in its central property: nothing an unauthenticated stream sends can alter peer state, because endpoints are synthesised from the kernel socket's own tuple (`wg_tcp.c:3237–3440`), promotion requires a Noise-authenticated message carrying a device-local connection id, and the configured listen port is never replaced by an ephemeral one (`wg_tcp.c:630–693`). The limits around the model are where the review found trouble: the sizing and refresh rules of the pending table (RVPN-01-603), the unbounded lifetime of authenticated-but-unpromoted entries (RVPN-01-604), the device-wide bypass of MAC1, cookies and the rate limiter (RVPN-01-605), and the assertion that bypass exposes (RVPN-01-601). The design document already lists the cost defence as remaining work; Red Team Ltd regards it as a prerequisite for exposing the listener to a hostile network. Commit 8baef74 removed the assertion and restored `mac1` validation (RVPN-01-601 fixed, 605 partially fixed); the cookie challenge and the rate limiter remain disabled for TCP-mode devices, and the admission limits are unchanged.

Per-connection memory and table limits. Per-stream state is bounded (parser buffer at the maximum record size, send queue at 1,024 frames, pending and tracked tables at 128 and 1,024 entries, a fixed 128-slot throttle table that allocates nothing per source), and the accounting under `tcp_connection_list_lock` is consistent with underflow guards. The bounds are sized for a laboratory rather than for the intended population, and recla-

mation pays one RCU grace period per entry (wg_tcp.c:4361); both points are covered in RVPN-01-603 and RVPN-01-604.

Socket callbacks and worker contexts. This is the strongest part of the delta. The three callbacks read sk_user_data exactly once under the read side of sk_callback_lock, keep the saved original callbacks in a wrapper that the peer also retains, and never sleep (wg_tcp.c:2407–2642, 3877–3979). Installation and removal take the callback lock write side before tcp_lock, verify the exact directional owner, and publish the ownership flag last (wg_tcp.c:3981–4150). Each published wrapper holds a module reference, so a callback can never run into unloaded text. Reads and writes run on workqueues, with a scheduled-flag protocol that closes the lost-wakeup window (wg_tcp.c:3849–3875). The one context violation found is the netns exit path on Linux 7.2 (RVPN-01-608).

Locking and RCU. The lock hierarchy tcp_socket_lock (mutex) above tcp_lock above the read, write and send-queue spinlocks is applied consistently, lockdep_assert_held() is used at the right places, and configured peers are always locked before provisional peers so that promotion never takes two owner mutexes in an address-dependent order (wg_tcp.c:4453–4458). The provisional list is a genuine RCU list with list_add_tail_rcu, list_del_rcu and a grace period before destruction. The defect was the use of RCU traversal on a list that is not RCU-protected (RVPN-01-602); dead code repeated the pattern (RVPN-01-617). Both were removed in the remediation pass, and no RCU traversal of peer_list remains.

Refcounting and teardown. Teardown is the most intricate part of the module and it is handled with discipline: every removal path claims an exact socket under tcp_lock, drains the stream workers, waits out in-flight callbacks with a write-lock pair on sk_callback_lock, detaches the wrapper and only then releases the socket once (wg_release_peer_socket_locked, wg_tcp.c:1054–1113). Failure of a detach invariant quarantines the peer with a permanent reference rather than freeing memory a callback might still reach. wg_peer_remove() and wg_peer_remove_all() stop TCP state before peer_make_dead(), and device stop and destruct drain provisional entries twice around the listener release. No use-after-free or double release was identified in the live paths. The unused wg_clean_peer_socket() would not meet this standard if it were ever called (RVPN-01-617).

skb handling, framing and partial reads. Every record-driven allocation is preceded by wg_check_potential_header_validity
↪ () (wg_tcp.c:3216–3235), which enforces the type, the flag mask, the WireGuard minimum and the maximum record size, and resynchronisation uses the same validator while preserving a bounded suffix. Headroom reserved for the synthetic headers is sufficient, leftover bytes are copied before the buffer is trimmed, and short writes advance the exact framed byte sequence. The header checksum is not a security control and should not be described as one (RVPN-01-606); the fragment header leaks inner metadata and should go (RVPN-01-610).

Roaming and endpoint authentication. Authenticated roaming updates only the address, keeps the operator's listen port, and advances a monotonic generation so that an older stream cannot revert a newer observation (wg_tcp.c:662–693, 4421–4423). That design is correct. The consequences of data-driven promotion for a displaced carrier, and the absence of liveness timeouts, are recorded in RVPN-01-618; the dropped first handshake response in RVPN-01-609 is a functional defect of the same promotion sequence.

Cookie and rate-limiter interaction. In TCP mode the module neither validates MAC1 nor issues or consumes cookies, and the rate-limiter tables are never consulted, for either carrier (RVPN-01-605). The reply helpers still route through UDP, so enabling the mechanism requires the same-stream reply path that the design document stages.

TCP-over-TCP behaviour. Per-peer isolation, TCP_NODELAY, nonblocking writes with write-space wakeups and a bounded drop-newest queue are the right building blocks, and the meltdown study in docs/TCP_MELTDOWN.md is measured rather than asserted. What is missing is stall detection and a user timeout (RVPN-01-618) and the MTU accounting for the extra 28 bytes of carrier overhead (RVPN-01-617).

Netlink and the generated policy. Both netlink implementations check CAP_NET_ADMIN in the creating namespace for port, mark and transport changes (kernel/netlink.c:505, 1505), validate the transport value, refuse live transport and port changes with EBUSY, and keep the upstream attribute policies intact. The generated policy declares the new attribute with the right type. The attribute numbering, the hand-edited generated file and the full-dump capability check are recorded in RVPN-01-616. Version 1.1 adds RVPN-01-620: the EBUSY guard for a live listen-port change in TCP mode exists only in the pre-7.0 branch, so the documented contract does not hold on the fleet's 7.x kernels.

Peer table scaling for ten million peers. The hashtables are scaled sensibly (`kernel/peerlookup.h:17–37`), the public-key table is keyed with a random SipHash key, and the index table is keyed by locally generated random indices, so neither offers an attacker-chosen collision surface. The per-peer resource model does not scale: two rescuer-backed workqueues per peer (RVPN-01-607) make the target unreachable, the accept-path walk (RVPN-01-602) makes every connection cost proportional to the population, and the admission limits (RVPN-01-603) are three orders of magnitude below it. Upstream per-peer structures such as the per-CPU destination cache and the NAPI instance were never designed for this count either, which the maintainers should measure before committing to the figure (Appendix D).

Debug tracing. Production builds compile out `wg_dbg()`, and the DKMS build passes no diagnostic flags. The verbose level prints keys and plaintext and uses `%px` throughout (RVPN-01-612); production paths still carry unratelimited logging and one raw pointer (RVPN-01-611). The fault-injection parameters are correctly confined to `DEBUG` plus an explicit flag. Commits cc6ea2f and c7c0260 removed the key and plaintext dumps, replaced every `%px` and ratelimited the remote-driven sites (RVPN-01-611 and 612 fixed); the build guard and module marker recommended in 612 were not added.

UAPI shim and ABI across 6.12 to 7.2. Layering the extension over the kernel's own `<uapi/linux/wireguard.h>` is the right choice and avoids the duplicated-enum failure the shim's comment describes. The dynamic allowed-IP values match the ones mainline adopted in 7.0. The transport attribute's number is the one open point (RVPN-01-616).

Version guards and the WQ_PERCPU probe. The guards for `sockaddr_unsized`, `timer_container_of`, the BLAKE2s API, the UDP tunnel API, the pernet `exit_rtnl` hook and the generated netlink split are coherent and cover the declared range. The header-path heuristic behind `HAVE_WQ_PERCPU` misdetects on Debian cloud and real-time flavours and on Ubuntu (RVPN-01-615).

The `wg` tool fork. The delta is small, the parsing is strict, and the fork fails closed against kernels and userspace implementations that do not support the attribute. The native build and the sanitiser harness (Appendix A-10 to A-12) found no memory-safety defect in 200,000 mutated inputs. The `showconf` portability regression was fixed in 7eab4e2 and the rebuilt fork again passed the harness (A-20); the dump-based capability check and the attribute numbering remain (RVPN-01-616).

Debian packaging and DKMS. The DKMS source package builds reproducibly from the shipped sources and the Ubuntu archive scripts verify release, architecture, kernel ABI and SHA-256 sums before installing. The maintainer scripts install into and delete from `/usr/local`, depend on `build-essential`, bypass `dh-dkms`, do not restrict the kernel range and ship no module signature (RVPN-01-614). No supply-chain compromise vector was found in the scripts themselves: nothing is downloaded, and the patch generation script works from clean upstream checkouts.

`tools/scale_test.py`. The script is straightforward and safe, but it measures UDP-mode peers only and therefore does not support any claim about TCP mode (RVPN-01-607).

Conformance. Against the WireGuard protocol, the handshake, replay and rekey rules are preserved, while the cookie mechanism is absent in TCP mode and the simultaneous-initiation tie-break is a deviation (RVPN-01-605, RVPN-01-617). Against the kernel's locking, RCU, workqueue and printk documentation, the module conforms in its callback and teardown design and departs in the RCU traversal, the netns exit path, the per-peer rescuer workqueues and the unratelimited logging. Against Debian Policy and DKMS conventions, the package needs the corrections in RVPN-01-614. The detailed checklist is in Appendix C.

What held up well. The exact-socket ownership and detach protocol; the authenticated-promotion design with monotonic connection generations; the synthesis of endpoints from socket state rather than stream contents; the bounded, validated framing parser; the keyed and randomised hashtables; the netlink capability checks and fail-closed tool behaviour; the reproducible patch and the checksummed binary archives; the candour of the design document about open controls; the absence of any change to the cryptographic code; and the speed and precision of the remediation pass, which closed every item that was local to fix within the engagement window without introducing a regression visible to review.

Verdict. At 97c267e the module is suitable for the controlled deployments its own documentation describes, not yet for direct exposure to a state-level adversary and not for the ten-million-peer population it names. The remediation pass closed the two High items that were local to fix (RVPN-01-601, 602) and the hygiene items

(611, 612, 613); the four High items that define exposure at scale (603, 604, 606, 607), the two Medium items that affect reliability and connect latency (608, 609) and the partial cookie work (605) remain. Red Team Ltd recommends the following order of work:

1. Compile, load and exercise the remediated module on Linux 6.12 and 7.2 hosts, including the netns suites; this is the owed confirmation for every Fix Note in this report.
2. Replace the per-peer workqueues with the device workqueue and re-measure in TCP mode (RVPN-01-607).
3. Re-size and harden the admission tables: population-scaled caps, prefix-based source counting, deadlines for authenticated-but-unpromoted entries, batched reclamation, exported counters (RVPN-01-603, 604).
4. Decide and document the exposure model; front the listener with the obfuscated transport or add header obfuscation (RVPN-01-606).
5. Complete the cookie work with a same-stream cookie reply and rate limiting for TCP-mode devices, and offer a TCP-only mode that does not bind the UDP companion socket (RVPN-01-605).
6. Fix the 7.2 netns exit ordering (RVPN-01-608), send pre-session replies on the originating stream (RVPN-01-609), and hoist the listen-port guard into a shared helper (RVPN-01-620).
7. Address the remaining Low and Info items in a packaging and hygiene pass: Policy conformance, the build probe, carrier timeouts, the UAPI numbering, the open parts of 617, the build guard and module marker of 612, and the redaction list.

Red Team Ltd thanks the RostamVPN maintainers for the complete source access, the prepared upstream baselines that made exact delta isolation possible, and the thorough design documentation, which materially improved the precision of this review.

Appendix A: Dynamic test log

All checks ran on the assessment host (macOS on Apple Silicon) from the repository root; scratch files were kept under `/tmp/rvpn01/wp6`. No service was started and no network connection was made, so no process identifiers were recorded. No Linux kernel build environment exists on the host; the module was neither compiled nor loaded.

A-01 Delta isolation. Command: `diff -ruN /tmp/wgtcp-base/linux/drivers/net/wireguard kernel > /tmp/rvpn01 ↪ /wp6/kernel-delta.diff; diff -ruN /tmp/wgtcp-base/wgt-full/src tools > /tmp/rvpn01/wp6/tools-delta.diff`, followed by per-file `diff | grep -c '^>'` counts. Result: kernel delta 12,056 diff lines (+8,439 / -374 code lines; new files `wg_tcp.c` 4,987, `wg_tcp_debug.c` 1,207, `wg_tcp.h` 88, `wg_tcp_debug.h` 90, `wireguard_tcp_uapi.h` 56; largest modified files `netlink.c` +1,073, `device.c` +429, `receive.c` +410, `send.c` +243, `peer.c` +192, `socket.c` +157, `allowedips.c` +111); tools delta 559 diff lines over nine files plus the new `scale_test.py`. Pass.

A-02 Redaction scan. Commands: `grep -rnoE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' .` filtered for private, documentation and version-like values; `grep -rnoiE '2[0-9a-f]{3}:[0-9a-f:]+' .` excluding `2001:db8`; `grep -rnoE ↪ '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[a-z]{2,}' .`; a recursive grep for absolute home-directory paths; `grep -↪ rnoE '\b[A-Za-z0-9+/]{42}[AEIMQUYcgkosw048]=' .` with each hit compared against the upstream manual-page example keys; `grep -rniE 'client_id|oauth|api[_-]?key|BEGIN .* PRIVATE' .`. Result: two public addresses in `tools/man/wg.8:195,205` (inherited from upstream, present in the baseline); mailboxes at `tools/scale_test.py:2 ↪ , debian/changelog:9,19,29`, `debian/control:4`, `perf-test/meltdown/INVESTIGATION_STATUS.md:260,263`, `perf-test/ ↪ harness/refresh-report.ps1:95`; two non-upstream public keys in `docs/TUNNEL_CONFIG.sh` and `docs/NODE_README ↪ .md`; no OAuth identifiers, private keys, CDN hostnames or `/Users` paths. Reported as RVPN-01-619.

A-03 Shell syntax checks. Command: `for f in debian/postinst debian/prerm scripts/generate-big-patch.sh ↪ scripts/build-ubuntu-binary.sh scripts/install-ubuntu-binary.sh; do sh -n $f; bash -n $f; done`. Result: all pass under `bash -n`; all pass under `sh -n` except `scripts/install-ubuntu-binary.sh`, which uses process substitution and carries a bash shebang (expected). Pass.

A-04 Python syntax check. Command: `python3 -I -c 'import ast,sys; src=open(sys.argv[1]).read(); ast. ↪ parse(src); print(src.count("\n"))' tools/scale_test.py`. Result: `117`, no error. Pass.

A-05 WQ_PERCPU probe test. Command: `for p in PATHS; do echo "$p" | sed 's/-[a-z0-9_]*$/-common/'; ↪ done` over the six header paths named below, using the expression from `kernel/Makefile:19`. Result: `...+`

↪ deb13-amd64 and ...+deb13-arm64 and linux-headers-7.2.1-1-amd64 map to the correct -common sibling; ...+deb13
↪ -cloud-amd64 maps to ...-cloud-common, ...+deb13-rt-amd64 to ...-rt-common, and linux-headers-6.14.0-29-
↪ generic to ...-29-common, none of which exist. Fail for cloud, rt and Ubuntu layouts (RVPN-01-615).

A-06 Packaging rules parse. Command: make -n -f debian/rules -p >/dev/null. Result: exit 0 (rules parse OK
↪); grep -c '@DKMS_VER@' debian/dkms.conf.in debian/postinst debian/prerm returned one placeholder in each, matching the substitutions in debian/rules:26,32-33. Pass.

A-07 Framing model. Command: python3 -I /tmp/rvpn01/wp6/py/a4_framing.py (a 60-line model of wg_header_checksum(), wg_tcp_build_frame() and wg_check_potential_header_validity()). Result: constant twelve-byte prefixes 0000009c0000b62501000000 (initiation), 000000640000a92502000000 (response), 000000480000
↪ aca503000000 (cookie reply), 000000280000a0a504000000 (keepalive); random eight-byte windows accepted 0 of 2,000,000 times; attacker-minted headers for 1,000-byte and maximal fragmented records validate. Used in RVPN-01-601, RVPN-01-605, RVPN-01-606.

A-08 Admission model. Command: python3 -I /tmp/rvpn01/wp6/py/a5_admission.py (discrete-time model, 100 ms steps, of the limits at wg_tcp.c:250-257). Result: sources needed to fill the device-wide pending cap: 16
↪ ; sustained connection rate needed to hold the cap: 4.3 conn/s; 16-source attacker refilling immediately:
↪ legitimate admissions 0/3001 over 300 s; time to fill 1024 tracked entries at 32/s: 32 s. Used in RVPN-01-603, RVPN-01-604.

A-09 Netlink attribute numbering. Command: python3 -I /tmp/rvpn01/wp6/py/a6_netlink_abi.py. Result: tools enum WGDEVICE_A_TRANSPORT=9, kernel shim __WGDEVICE_A_LAST=9, consistent; [WGDEVICE_A_TRANSPORT] = { .type =
↪ NLA_U8 } present in kernel/generated/netlink.c and kernel/netlink.c; WG_TRANSPORT_UDP=0, WG_TRANSPORT_TCP=1 in both headers. Pass.

A-10 Native build of the tool fork. Command: cp -a tools /tmp/rvpn01/wp6/tools-build && make -C /tmp/
↪ rvpn01/wp6/tools-build -j4 && /tmp/rvpn01/wp6/tools-build/wg --version. Result: twelve objects compiled and linked with Apple clang 21.0.0 without warnings; banner wireguard-tools v1.0.20260223. Pass.

A-11 Tool runtime checks. Commands (from the build directory): ./wg set wg0 transport tcp, ... transport
↪ udp, ... transport foo, ... transport, ./wg set wg0 listen-port 51820 transport TCP, ./wg set wg0 peer KEY
↪ transport tcp (with the upstream manual-page example key), ./wg show wg0 transport, and ./wg setconf
↪ wg0 FILE with Transport = tcp, TransportMode = Tcp, and Transport inside [Peer]. Results: Unable to modify
↪ interface: Operation not supported on socket for TCP on the userspace path (fail-closed), No such file
↪ or directory for UDP, Transport is neither udp nor tcp: 'foo', Invalid argument: transport for the missing value and for transport after peer, Line unrecognized: 'Transport=tcp' for the [Peer] placement. Pass.

A-12 Sanitiser harness. Commands: CFLAGS="-O1 -g -fsanitize=address,undefined -fno-omit-frame-pointer
↪ " LDFLAGS="-fsanitize=address,undefined" make -j4 in the build copy; clang -O1 -g -fsanitize=address,
↪ undefined -std=gnu99 -D_GNU_SOURCE -I. -o /tmp/rvpn01/wp6/harness /tmp/rvpn01/wp6/harness.c config.o
↪ encoding.o (the same objects tools/fuzz/Makefile links for its config fuzzer); ASAN_OPTIONS=detect_leaks=0 /
↪ tmp/rvpn01/wp6/harness. Result: checks=13 failures=0; fuzz: 200000 mutated lines, accepted=11266 rejected
↪ =188734, no sanitizer reports; sanitiser report lines: 0. A first revision of the harness freed ctx.device after a failed config_read_line(), which the fork already frees at tools/config.c:553-554; AddressSanitizer reported that use-after-free in the harness itself and the harness was corrected. Pass.

A-13 Grep audits. Commands: grep -nE 'wg_tcp_queuepkt\(|wg_clean_peer_socket\(|wg_find_peer_by_endpoints
↪ \(' kernel/*.c (call sites: one, inside dead code); grep -nE '(printk|pr_err|pr_warn|WARN)' kernel
↪ /*.c | grep -E '%px|%p[^IiSsh]' | grep -v wg_dbg (one hit, wg_tcp.c:2434); grep -cE 'pr_err\(|printk\(
↪ KERN_ERR|pr_warn\(|printk\(KERN_WARNING' kernel/wg_tcp.c (61) against grep -c ratelimited (5); grep -nE '
↪ list_for_each_entry_rcu\(.*peer_list' kernel/*.c (wg_tcp.c:1364, 1460) against grep -nE 'list_(del_init|
↪ add_tail)\(&peer->peer_list' kernel/*.c (peer.c:148, 189, 346); grep -n alloc_workqueue kernel/*.c (per-peer at peer.c:126, 133); grep -n reuseport kernel/*.c (wg_tcp.c:1700); grep -nE 'LINUX_VERSION_CODE|HAVE_WQ_PERCPU
↪ ' kernel/*.c kernel/*.h kernel/generated/* (34 guard sites across cookie.c, device.c, device.h, netlink.c, noise.c, timers.c, socket.c, wg_tcp.c, wg_tcp_debug.c, wireguard_tcp_uapi.h, generated/netlink.[ch]). Used throughout.

A-14 Patch header check. Command: head -12 BIG-WireguardTCP-Patch. Result: Kernel baseline: Linux v7
↪ .2, Tools baseline: wireguard-tools a998407..., Target source commit: 581e597..., matching the brief's de-

ployed tag. Pass.

A-15 Report rendering. Command: `pandoc PENTEST-REPORT.md -f markdown -t html -o /tmp/rvpn01/wp6/report.html`. Result: pandoc 3.8.2.1 exited 0 without warnings and produced an HTML document of about 129 KB; the index entries matched the finding headings exactly and no table exceeded five columns. Pass.

A-16 Remediation delta isolation (version 1.1). Commands: baseline copies were placed under `/tmp/rvpn01/wp6/remed/old/kernel` and `.../old/tools`, the saved acd17ec deltas had their baseline path prefixes rewritten with `sed` to `kernel/` and `tools/`, and `patch -p0 --dry-run` followed by `patch -p0` applied them; `diff -ruN /tmp/wgtcp-base/linux/drivers/net/wireguard old/kernel | wc -l` returned 12,056, identical to the saved delta, confirming the reconstruction; then `diff -ru old/kernel kernel` and `diff -ru old/tools tools`. Result: 2,193 diff lines over `allowedips.c`, `device.c`, `device.h`, `netlink.c`, `peer.c`, `peer.h`, `receive.c`, `send.c`, `socket.c`, `socket.h`, `wg_tcp.c`, `wg_tcp.h` and `wg_tcp_debug.c`, and 14 lines over `tools/showconf.c`. Every hunk was read: `%px` to `%p` substitutions, removal of the key, plaintext and netlink-dump statements, the handshake-path reordering, the listener walk and dead-helper removals, the reuseaddr change, the ratelimited log sites, the macro and guard fixes, the removed fields and declarations, and the `showconf` condition. No change outside the eleven commits' stated scope was found. Pass.

A-17 Contract suite (version 1.1). Command: `PYTHONDONTWRITEBYTECODE=1 python3 -B -I -m unittest discover -s tests -t .` from the repository root. Result: `Ran 228 tests`, `FAILED (failures=4, errors=3)`, matching the seven pre-existing results the maintainers reported. Failing: `test_tcp_connect_waits_for_a_configured_endpoint` and `test_accepted_remote_address_is_initialized_and_validated` (contract text predates the `(const struct sockaddr *)` casts), `test_outbound_socket_uses_creation_namespace_and_device_mark` (contract text predates the `WG_KSA()` cast), `test_live_tcp_port_change_is_rejected_before_mutation` (genuine: the 7.0+ `set_port()` lacks the `EBUSY` guard, RVPN-01-620). Erroring: `test_unauthenticated_accepts_have_source_and_rate_limits` and `test_outbound_tuple_is_cached_after_connect_selects_it` (`index()` on contract strings that predate the casts), `test_udp_remains_zero_and_default_port_is_not_forced` (reads `include/uapi/linux/wireguard.h`, which the UAPI shim replaced). The four `__pycache__` directories under `tests/` and `perf-test/` carry modification times of 22:47 and 23:16 on 2026-10-14, before this run, and were not created by it. Pass with seven known results.

A-18 Regenerated patch re-verification (version 1.1). Commands: baseline copies laid out as `kernel/` and `tools/` under `/tmp/rvpn01/wp6/bigcheck`; `patch -p1 --dry-run < BIG-WireguardTCP-Patch` (Apple patch 2.0-12u11), then `patch -p1`, then `diff -ru` of the result against the current `kernel/` and `tools/`. Result: header names Linux v7.2, wireguard-tools a998407 and target commit 1e84c3a; the dry run and the application completed without a rejected hunk; the kernel comparison produced 0 diff lines and the tools comparison 0 diff lines once the `wg-quick/wg` symlink and `scale_test.py`, which the generator excludes by design, are set aside. Pass.

A-19 Grep re-audits (version 1.1). Commands as in A-13 repeated on the current tree. Results: `%px` occurrences in `kernel/`: 0; RCU traversals of `peer_list`: 0; definitions or calls of `wg_tcp_queuepkt`, `wg_find_peer_by_endpoints`, `wg_clean_peer_socket`, `wg_release_peer_tcp_connection`, `wg_free_peer_socket_data` and `wg_tcp_connection_retry_timer`: 0; `#ifdef CONFIG_IPV6`: 0; `sock_set_reuseaddr` at `wg_tcp.c:1520` and `1582`, `sock_set_reuseport`: 0; unratelimited `pr_err` and `printk` sites in `wg_tcp.c`: 56 against 9 ratelimited, none on a remote-driven path; key-dump statements: 0. Unchanged, as expected for open items: the admission constants (`wg_tcp.c:250-257`), the `entry->authenticated` exemption (`wg_tcp.c:4039`), the per-peer `alloc_workqueue()` calls (`peer.c:123`, `130`), `cancel_delayed_work_sync(&wn->fib_dispatch_work)` under `exit_rtnl` (`device.c:760`), the fragment header, the `/usr/local` paths in the maintainer scripts, the `sed` probe (`Makefile:19`), and the absence of `TCP_USER_TIMEOUT`. Pass.

A-20 Tool rebuild and harness on the remediated sources (version 1.1). Commands as in A-10 and A-12 on a fresh copy of `tools/`. Result: twelve objects built without warnings, `wg` 104,976 bytes, banner `wireguard-tools v1.0.20260223`; harness `checks=13 failures=0`, `200000 mutated lines, accepted=11266 rejected=188734`, sanitiser report lines 0. Pass.

A-21 Report rendering (version 1.1). Command: `pandoc PENTEST-REPORT.md -f markdown -t html -o /tmp/rvpn01/wp6/report.html`. Result: exit 0 without warnings; the index matched the twenty finding headings; no table exceeded five columns. Pass.

Appendix B: Dependency inventory and advisories

Component	Revision or version	Source and pinning	Advisories
Linux in-tree WireGuard (baseline for `kernel/`)	v7.2, commit 8d3ae59288f1	Pinned in `BIG-WireguardTCP-Patch`; DKMS builds the same source against each fleet kernel 6.12 to 7.2	None specific to the delta; upstream fixes after v7.2 must be rebased by hand
wireguard-tools (baseline for `tools/`)	a998407747005ea7, banner v1.0.20260223	Pinned in the patch header	None known
libmnl	Distribution package, unpinned	`Depends: libmnl-dev`; linked by the `wg` build in `postinst`	None known
Build toolchain	dkms, build-essential, pkg-config (distribution, unpinned)	`debian/control:11-15`	Policy issue, RVPN-01-614
Prebuilt archives	Ubuntu 24.04, kernel 6.8.0-136-generic, amd64 and arm64	`docs/downloads/`, SHA-256 sums in `SHA256SUMS.txt`, no detached signature	Unsigned module and tool; kernel outside the declared 6.12 to 7.2 range

No Rust, JVM or Swift dependency manager is used in WP6, so cargo audit, Gradle and SwiftPM checks do not apply. No SBOM in CycloneDX 1.7 or SPDX 3.0.1 form is present; `manifest.json` in the binary archives records the source revision and build time but no attestation, so SLSA provenance is absent. The patch generation script (`scripts/generate-big-patch.sh`) and the Ubuntu build script perform no downloads and build only from committed sources, which is the correct basis for reproducible provenance once an attestation is added. At 97c267e the regenerated patch (commit c38845b, target 1e84c3a) was re-applied to the baselines and reproduces the current `kernel/` and `tools/` exactly (Appendix A-18).

Appendix C: Standards conformance checklist

NIST SP 800-218 SSDF v1.1 practices, as applied to this work package:

Control	Status	Note
PO.1 Define security requirements	Partial	Design document states controls; no exposure model or scale requirement is written down
PO.3 Implement supporting toolchains	Partial	No sparse, checkpatch or sanitiser run in CI; fuzz harnesses exist upstream but are not run for the fork
PO.5 Secure environments for development	Not assessed	Outside the repository
PS.1 Protect all forms of code	Not assessed	Commit signing and branch protection not visible from the tree
PS.2 Verify software release integrity	Partial	SHA-256 sums published; no signatures on archives or module (RVPN-01-614)
PS.3 Archive and protect each release	Partial	Patch and manifest pin revisions; no SBOM or provenance attestation
PW.1 Design to meet security requirements	Partial	Pre-authentication defences deferred (RVPN-01-605), limits sized for a lab (RVPN-01-603)

Control	Status	Note
PW.2 Review the design	Partial	Design document exists; the ten-million-peer design was not reviewed for per-peer resources (RVPN-01-607)
PW.4 Reuse well-secured software	Pass	Cryptographic code is upstream WireGuard, unchanged
PW.5 Follow secure coding practices	Partial	RVPN-01-601, 602 and 611 fixed at 97c267e; RVPN-01-608 and 620 open
PW.6 Configure compilers and build tools	Partial	Debug gating by flags only (RVPN-01-612); fragile build probe (RVPN-01-615)
PW.7 Review human-readable code	Partial	Dead code and stale helpers remain (RVPN-01-617)
PW.8 Test executable code	Partial	Contract and netns tests exist; no framing fuzzing; scale test covers UDP mode only
PW.9 Configure software to secure defaults	Partial	MAC1 restored and reuseport removed at 97c267e; cookie challenge, rate limiter and UDP companion binding remain (RVPN-01-605)
RV.1 Identify and confirm vulnerabilities	Partial	No published security contact or disclosure process in the tree
RV.2 Assess, prioritise and remediate	Not assessed	No history available within scope
RV.3 Analyse root causes	Not assessed	No history available within scope

Kernel-module controls defined by Red Team Ltd for this work package and applied consistently:

Control	Status	Note
K1 Lock hierarchy documented and applied	Pass	Mutex above `tcp_lock` above leaf spinlocks; `lockdep_assert_held` used
K2 No sleeping under spinlocks or in callbacks	Pass	Sends and `synchronize_rcu` moved to process context
K3 RCU primitives only on RCU-protected structures	Pass	Unsafe `peer_list` traversal removed in e0cd9e4 (RVPN-01-602)
K4 Single release path per socket, wrapper and skb	Pass	Exact-socket claims; no double release found
K5 Teardown drains callbacks before freeing `sk_user_data`	Pass	Write-lock pair on `sk_callback_lock` before detach
K6 Work cancellation never waits under a lock the work takes	Fail	7.2 netns exit (RVPN-01-608)
K7 Every length used for allocation or copy is validated	Pass	`wg_check_potential_header_validity` before each record allocation
K8 Headroom and tailroom sufficient for synthetic headers	Pass	128 bytes plus `NET_IP_ALIGN` reserved
K9 No WARN or BUG reachable from untrusted input	Pass	Assertion replaced by a ratelimited drop in 8baef74 (RVPN-01-601)

Control	Status	Note
K10 Remote-driven logging ratelimited, no raw pointers	Pass	Ratelimited in c7c0260, `%px` removed in cc6ea2f (RVPN-01-611)
K11 Resource bounds scale with intended population	Fail	RVPN-01-603, 604, 607
K12 Workqueues shared, WQ_MEM_RECLAIM only where required	Fail	RVPN-01-607
K13 Netlink policies strict, capabilities checked	Pass	`CAP_NET_ADMIN` in creating netns; strict nested parsing
K14 UAPI numbering coordinated, generated files regenerated	Partial	`showconf` fixed in 7eab4e2; numbering and generated policy open (RVPN-01-616)
K15 Version guards cover the declared kernel range	Pass	6.12 to 7.2 guards coherent; probe weakness in RVPN-01-615
K16 Debug code compiled out and detectable	Partial	Key and plaintext dumps removed in cc6ea2f; no build guard or module marker (RVPN-01-612)
K17 Secrets never logged in any build	Pass	Key logging removed in cc6ea2f (RVPN-01-612)
K18 Netdev contract (MTU, headroom, open and stop ordering)	Partial	MTU overhead not applied (RVPN-01-617); open and stop ordering correct
K19 Network namespace lifecycle handled	Partial	Correct on 6.12 to 7.1; deadlock risk on 7.2 (RVPN-01-608)
K20 Packaging follows Debian Policy and DKMS conventions	Fail	RVPN-01-614
K21 Live control-plane changes are atomic or refused	Fail	7.0+ branch lacks the listen-port guard (RVPN-01-620)

Appendix D: Open questions for the maintainers

1. The owed confirmation for every Fix Note in this report: compile, load and exercise the remediated module at 97c267e on Linux 6.12 and 7.2 hosts, including the netns regression suites, and confirm that handshakes still complete with `mac1` validation restored. Code review and the contract suite are the only verification performed so far.
2. Is the WGTCP listener ever reachable from the public Internet in production, or only through the obfuscated transports of WP1, WP2 and WP7? The answer sets the residual severity of RVPN-01-606.
3. Is the companion UDP port firewalled on every fleet host in TCP mode, as `docs/TCP_TRANSPORT_DESIGN.md` advises? Without that rule RVPN-01-601 and RVPN-01-605 are reachable by spoofed datagrams.
4. Are fleet kernels booted with `panic_on_warn` or `kernel.warn_limit`? This decides whether RVPN-01-601 is a crash or a log flood.
5. What peer count has actually been provisioned on a TCP-mode interface, and what thread count and memory did the host show at that point (RVPN-01-607)?
6. Does the WP5 client library retransmit the handshake initiation sooner than the protocol's five-second timer on a new TCP carrier, masking RVPN-01-609?
7. Have the upstream per-peer structures (per-CPU destination cache, NAPI instance, timers) been measured at one million peers on the target hardware?
8. Which hosts in the fleet run Debian cloud or real-time kernel flavours or Ubuntu, where the `WQ_PERCPU` probe misdetects (RVPN-01-615)?
9. Is module signing with a fleet MOK available, and is Secure Boot enabled on fleet hosts (RVPN-01-614)?
10. Has the `wireguard.yaml` netlink specification been updated anywhere outside this repository for the transport attribute (RVPN-01-616)?

11. Is an SBOM or build attestation produced for the DKMS package or the Ubuntu archives in the release pipeline (Appendix B)?